



Programmieren

Falk Morgenstern
Leon Johann Brettin

Foliensatz basierend auf Unterlagen von Prof. Huhn, Prof. Meyer und weiteren.
Nur zur Verwendung in der Vorlesung „Programmieren“ an der Ostfalia.
Keine Weitergabe.



Programmieren

10. Graphische Benutzeroberflächen



Programmieren

10. Graphische Benutzeroberflächen

10.1 Einleitung und Übersicht



Einleitung

- Graphische Benutzeroberflächen unter Java können auf Basis der folgenden Bibliotheken gebildet werden
 - **java.awt**
 - AWT (abstract window toolkit) enthält die Grundklasse und die Ereignisverarbeitung
 - **javax.swing**
 - Swing enthält Fenster und Elemente für Benutzeroberflächen (GUI-Komponenten)
 - **javafx**
 - JavaFX ist ein Framework zur Erstellung von plattformübergreifenden Java-Applikationen bzw. Rich Internet Applications. JavaFX sollte potentiell Swing ablösen.



Einleitung

- Ziel:
 - Grundelemente von JavaFX verstehen.
 - Einfache graphische Benutzeroberflächen in JavaFX realisieren.
 - Beispiel für die Einbindung einer einfachen graphischen Benutzeroberfläche in eine Applikation nach dem MVC Pattern.
 - Beschreibung von Benutzeroberflächen mit FXML (nur rudimentär).
- Quellen
 - JavaFX Dokumentation: <https://openjfx.io/>
 - Steyer, JavaFX, Springer Verlag (als Online-Buch in der Bibliothek erhältlich)

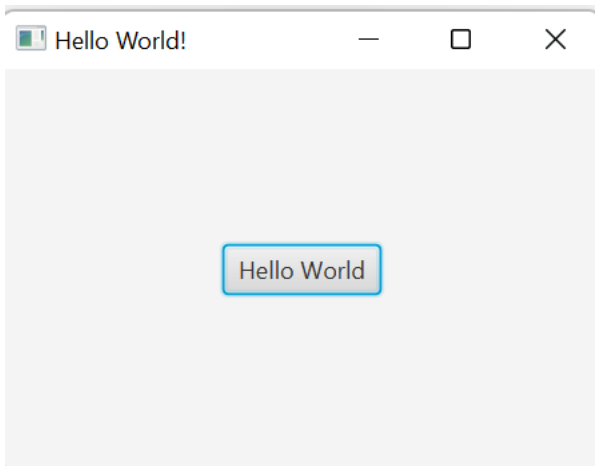


GUI-Elemente

- Eine JavaFX GUI besteht aus den folgenden Komponenten
 - Stage
 - Stage ist das Basisobjekt, in dem Applikationen wie auf einer Bühne angezeigt werden können (Top-Level JavaFX Container)
 - Scene
 - Szene (Applikation), die auf der Bühne dargestellt wird. Container für die Inhalte / Layouts / Widets, diese werden als hierarchischer Graph von Knoten dargestellt
 - Layout pane
 - Ermöglicht die Anordnung verschiedener GUI-Komponenten (auch als Layout Node bezeichnet)
 - Widgets
 - GUI-Elemente wie Textfelder, Buttons, Checkboxes oder Radiobuttons



Skelett für eine JavaFX-Anwendung



```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.stage.Stage;

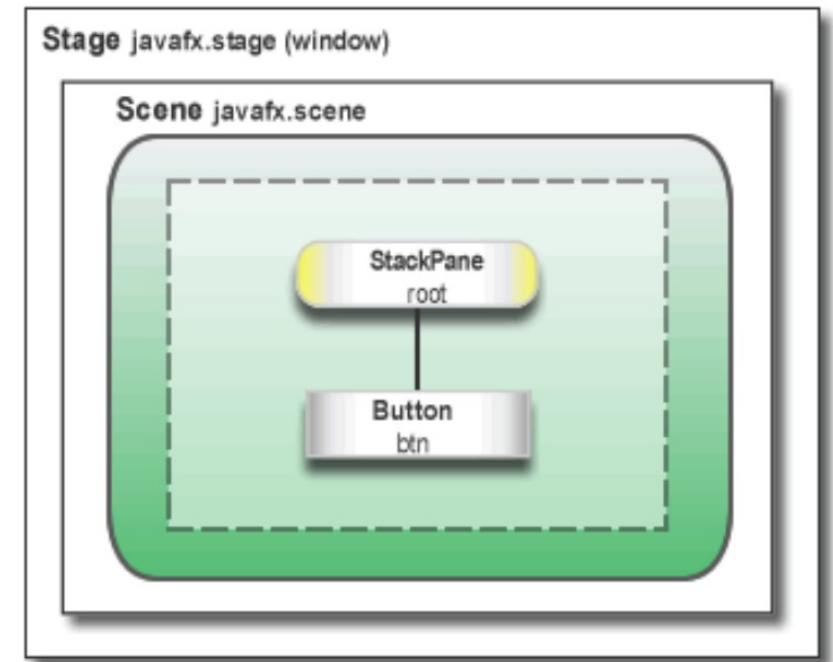
public class Main extends Application {

    public static void main(String[] args) {
        launch(args);
    }

    @Override
    public void start(Stage primaryStage) {
        Button btn = new Button("Hello World");
        StackPane root = new StackPane();
        root.getChildren().add(btn);
        Scene scene = new Scene(root, 300, 200);
        primaryStage.setTitle("Hello World!");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
}
```

Struktur einer JavaFX-Anwendung

- JavaFX GUIs werden abgeleitet von der abstrakten Klasse `javafx.application.Application`
- Deren abstrakte `start`-Methode wird überschrieben; diese bekommt eine `Stage` als Parameter. In der `start`-Methode wird eine `Scene` erzeugt.
- Der Inhalt der `Scene` wird in Form eines Baums als sogenannter `Scene Graph` dargestellt. Darin werden die GUI-Komponenten organisiert.

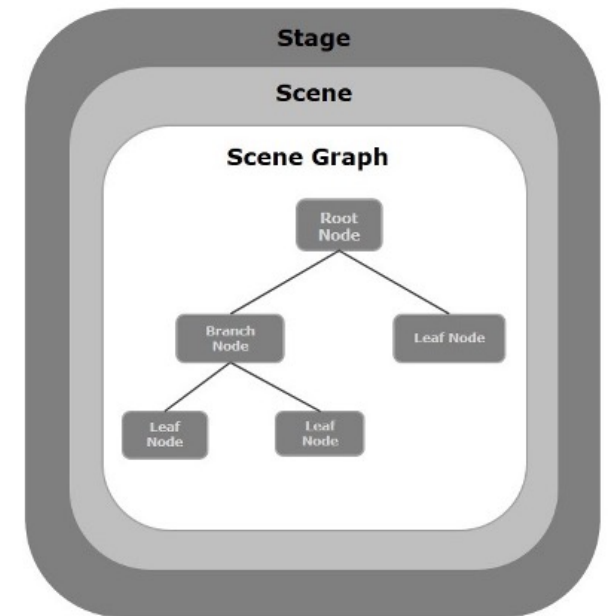


Quelle:

http://docs.oracle.com/javase/8/javafx/get-started-tutorial/hello_world.htm

Struktur einer JavaFX-Anwendung

- Stage
 - Enthält alle Objekte der Anwendung
 - `primaryStage` wird von der Plattform erzeugt und der `start`-Methode übergeben
 - Zur Darstellung: `show`-Methode aufrufen
- Scene
 - Enthält Inhalt der Anwendung (die Inhalte des Szenengraphen)
 - Erzeugung über Konstruktor z.B. mit Wurzelknoten, Breite, Höhe als Parameter
 - Scene-Objekt wird nur einer Stage zugeordnet
- Scene Graph
 - hierarchischer Graph, der die Inhalte einer Scene enthält
 - enthält i.a. Wurzelknoten, Verzweigungsknoten und Blattknoten
- Node
 - visuelles / graphisches Element eines Scene-Graphen



Quelle: https://www.tutorialspoint.com/javafx/javafx_application.htm

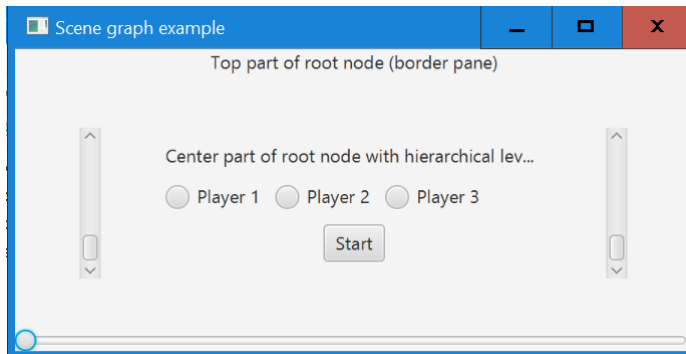


Schritte zur eigenen JavaFX-Anwendung

- Eigene Anwendungsklasse schreiben, die von `Application` abgeleitet ist
 - `Application`: abstrakte Klasse; Einstiegspunkt für JavaFX-Anwendungen
- Abstrakte Methode `start` von `Application` überschreiben
 - Die `primaryStage` wird von der Plattform erzeugt und der `start`-Methode als Parameter übergeben
- Scene Graph mit gewünschten Elementen (Nodes) erzeugen
- Scene (ggf. mit den erforderlichen Dimensionen Breite und Höhe) erzeugen und den Wurzelknoten des Scene Graphen hinzufügen
 - Als Parameter des Scene-Konstruktors oder über die `setRoot()`-Methode
- Scene der `primaryStage` zuordnen und `primaryStage` darstellen (`show`)
- `main`-Methode schreiben, die die Methode `launch` aufruft
- `launch`-Methode ruft die `start`-Methode von `Application` auf



Scene Graph: weiteres Beispiel



```
public void start(Stage primaryStage) {
    BorderPane root = new BorderPane();
    Label topLabel = new Label("Top part of root node (border pane)");
    root.setTop(topLabel);
    ScrollBar leftScrollBar = new ScrollBar();
    leftScrollBar.setOrientation(Orientation.VERTICAL);
    root.setLeft(leftScrollBar);
    Slider slider = new Slider();
    root.setBottom(slider);
    VBox vbox = new VBox();
    root.setCenter(vbox);
    Label label = new Label("Center part of root node with hierarchical
                             levels");

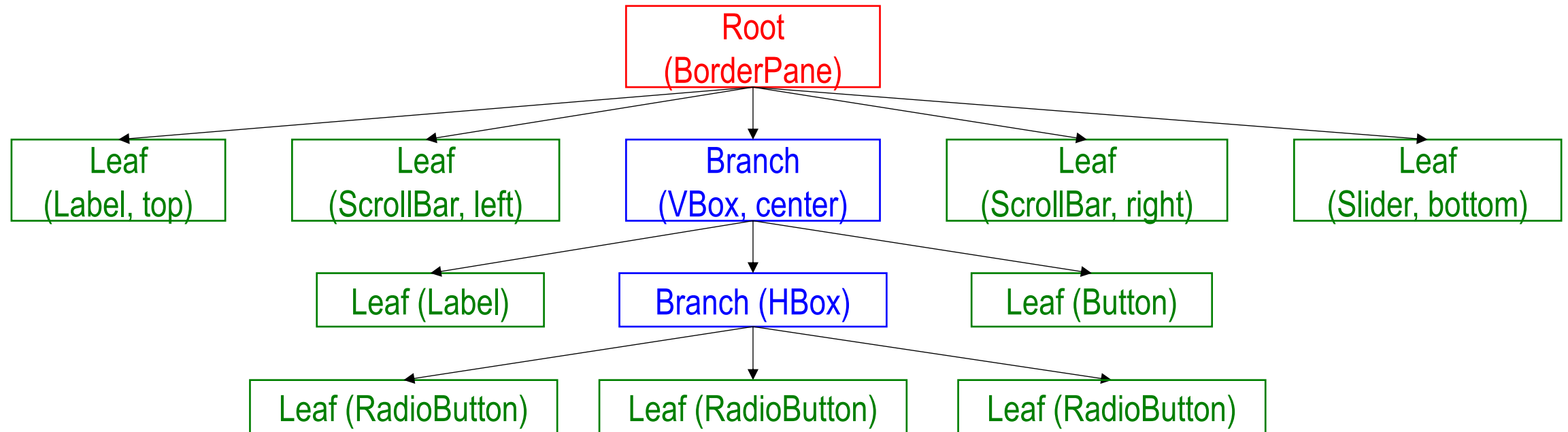
    HBox hbox = new HBox();
    Button button = new Button("Start");
    RadioButton radioButton1 = new RadioButton("Player 1");
    RadioButton radioButton2 = new RadioButton("Player 2");
    RadioButton radioButton3 = new RadioButton("Player 3");
    hbox.getChildren().add(radioButton1);
    hbox.getChildren().add(radioButton2);
    hbox.getChildren().add(radioButton3);
    vbox.getChildren().add(label);
    vbox.getChildren().add(hbox);
    vbox.getChildren().add(button);
    Scene scene = new Scene(root, 450, 200);
    primaryStage.setScene(scene);
    primaryStage.show();
}
```



Scene Graph: weiteres Beispiel

```
public void start(Stage primaryStage) {  
    BorderPane root = new BorderPane();  
    Label topLabel = new Label("Top part of root node  
                                (border pane)");  
  
    root.setTop(topLabel);  
    ScrollBar leftScrollBar = new ScrollBar();  
    leftScrollBar.setOrientation(Orientation.VERTICAL);  
    root.setLeft(leftScrollBar);  
    Slider slider = new Slider();  
    root.setBottom(slider);  
    VBox vbox = new VBox();  
    root.setCenter(vbox);  
    Label label = new Label("Center part of root node with  
                            hierarchical levels");  
  
    HBox hbox = new HBox();  
    Button button = new Button("Start");  
    RadioButton radioButton1 = new RadioButton("Player 1");  
    RadioButton radioButton2 = new RadioButton("Player 2");  
    RadioButton radioButton3 = new RadioButton("Player 3");  
    hbox.getChildren().add(radioButton1);  
    hbox.getChildren().add(radioButton2);  
    hbox.getChildren().add(radioButton3);  
    vbox.getChildren().add(label);  
    vbox.getChildren().add(hbox);  
    vbox.getChildren().add(button);  
    Scene scene = new Scene(root, 450, 200);  
    primaryStage.setScene(scene);  
    primaryStage.show();  
}
```

Scene Graph: weiteres Beispiel



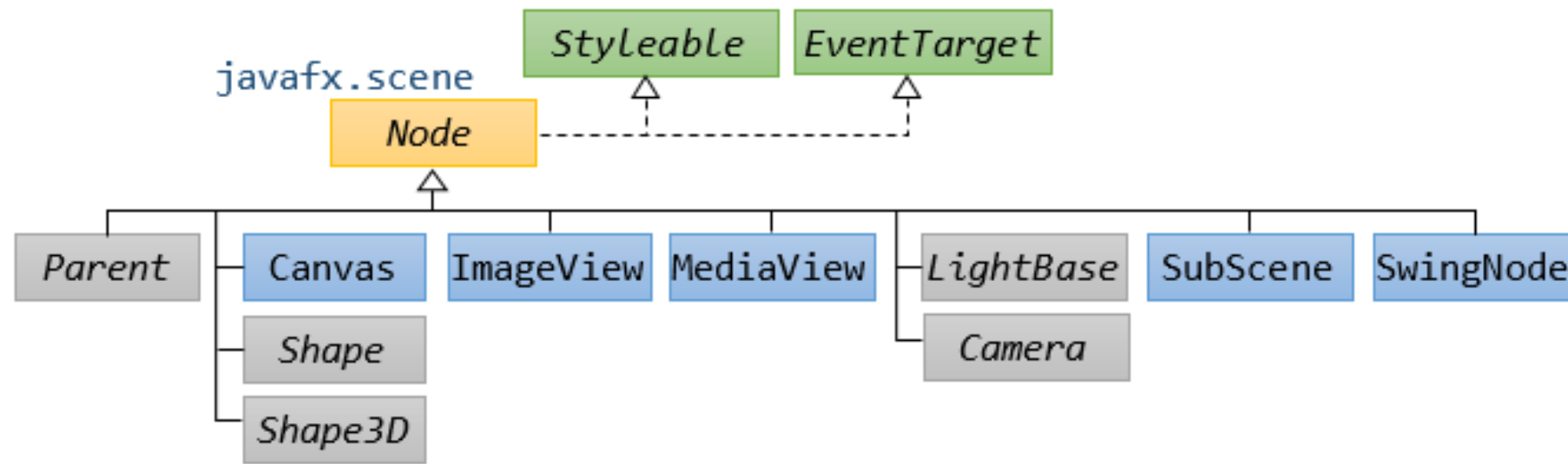


JavaFX Node

- Basisklasse für alle Elemente im Scene-Graphen
- Wichtige Unterklassen von Node (Auswahl):
 - Parent: alle Nodes, die ihrerseits Kindelemente haben können
 - Shape: Oberklasse geometrischer Formen (2D, 3D): Kreis, Rechteck, Polygon ...
 - Canvas: ermöglicht Zeichnen zweidimensionaler Grafiken (über Canvas-API)
 - ImageView: ermöglicht Darstellung von Bildern
 - MediaView: zur Integration von Audio- und Videokomponenten
- Wichtige Unterklassen von Parent (Auswahl):
 - Region: z.B. LayoutPanels und Controls für Benutzerinteraktion (später), Chart, Axis
 - Group: Kollektiver Knoten, enthält ObservableList von Kindknoten
- Eigenschaften (Auswahl):
 - Transformationen (Translation, Rotation, Skalierung)
 - Bounding Rectangle (umgebendes Rechteck)
 - CSS (Cascading Style Sheet)

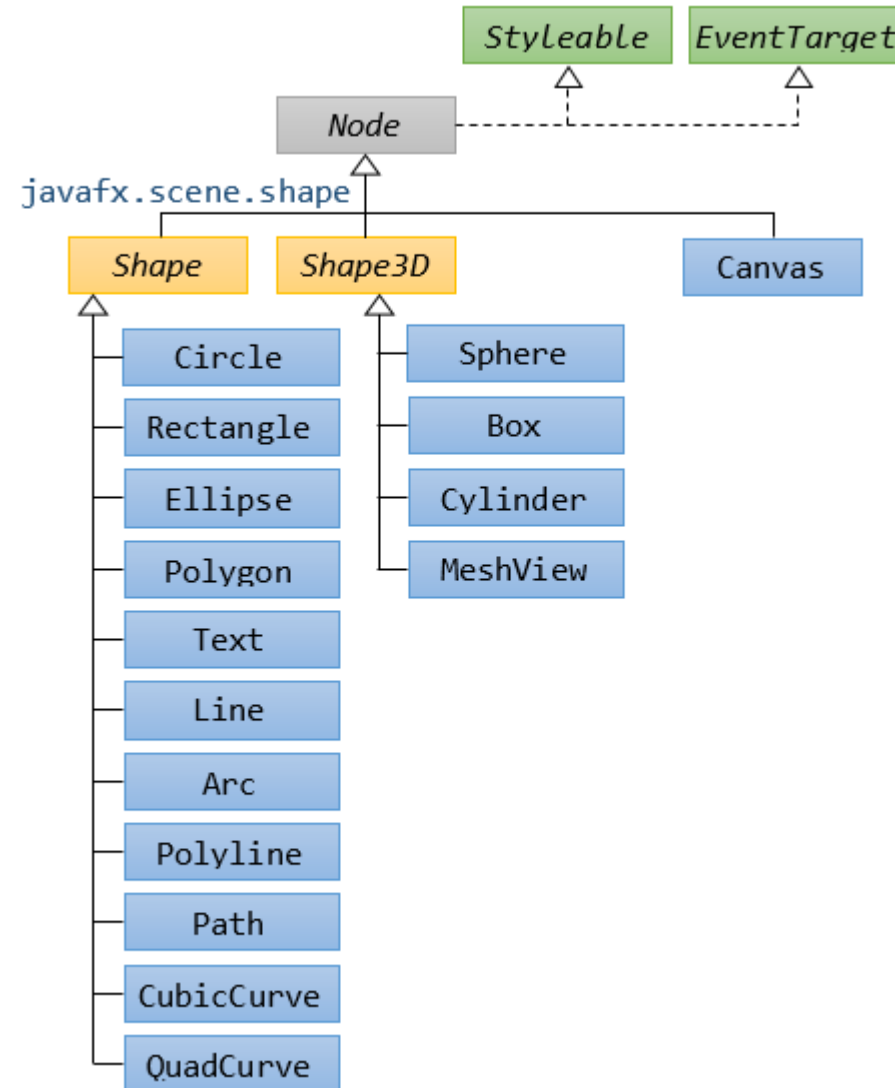


JavaFX Node



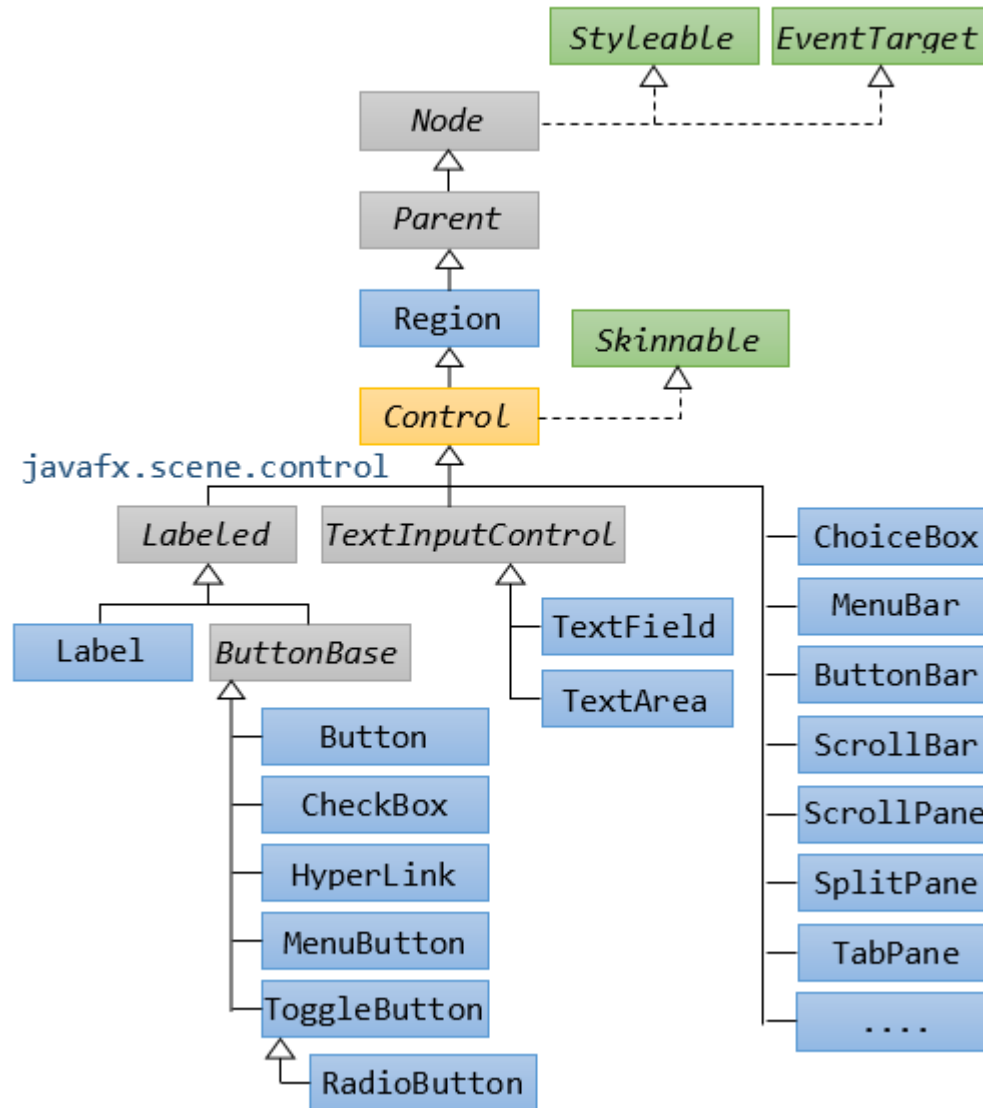


JavaFX Node



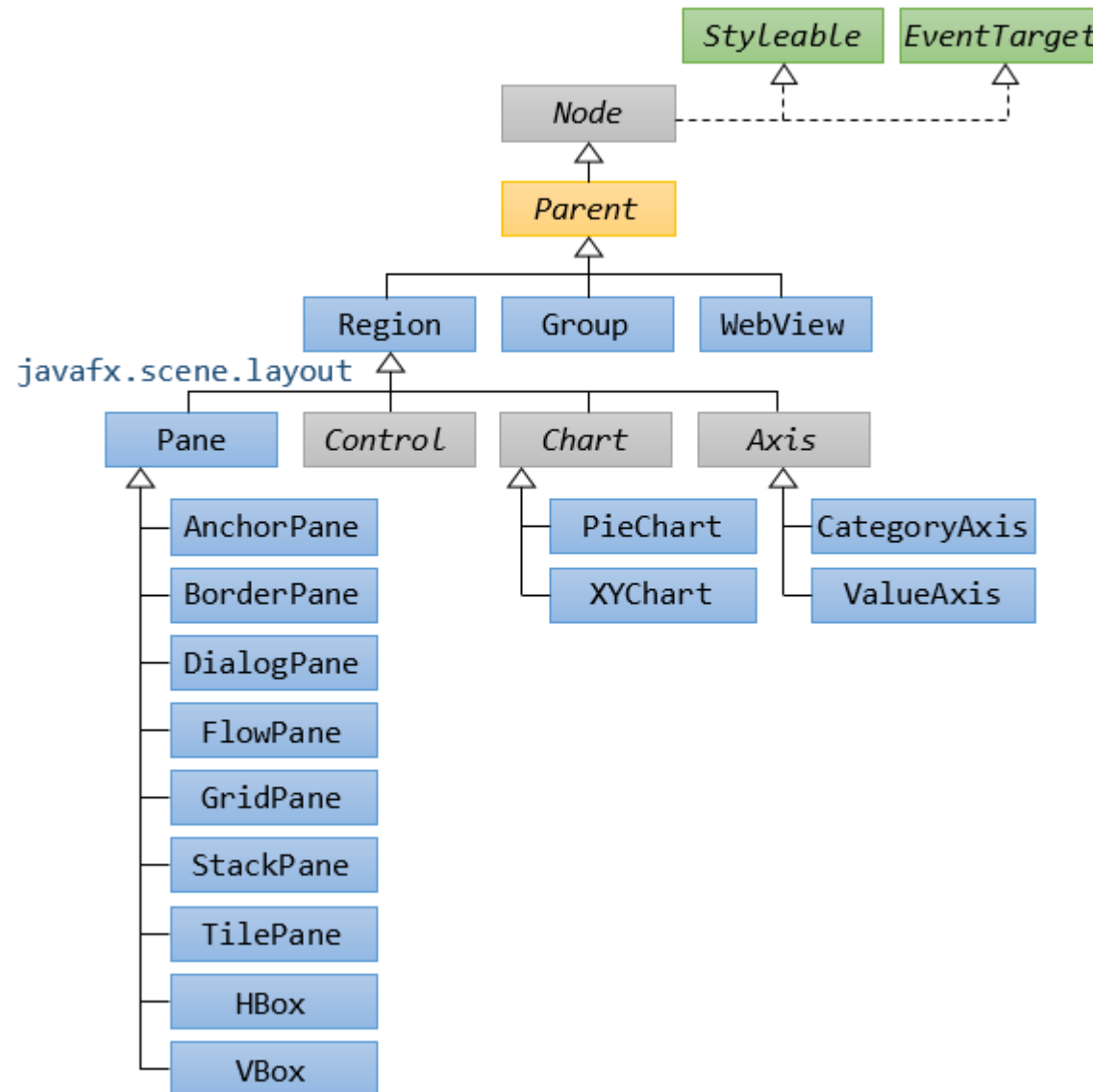


JavaFX Node





JavaFX Node





Programmieren

10. Graphische Benutzeroberflächen

10.2 Layout Panes



Layout Panes

- Zur Gruppierung mehrere GUI-Elemente verwendet man **Layout Panes**.
- Die Anordnung und Gestaltung der einzelnen GUI-Komponenten kann über die Layout Panes konfiguriert werden.
- Layout Panes dienen zum einfachen Aufbau und Management klassischer Layouts wie Zeilen, Spalten, Stapel usw.
 - Man könnte die Knoten prinzipiell auch einzeln (ohne Layout Panes) programmieren
 - Vorteil von Layout Panes: Wenn dessen Größe verändert wird, werden die einzelnen Komponenten automatisch repositioniert und skaliert
- Layout Panes kann man verschachteln.



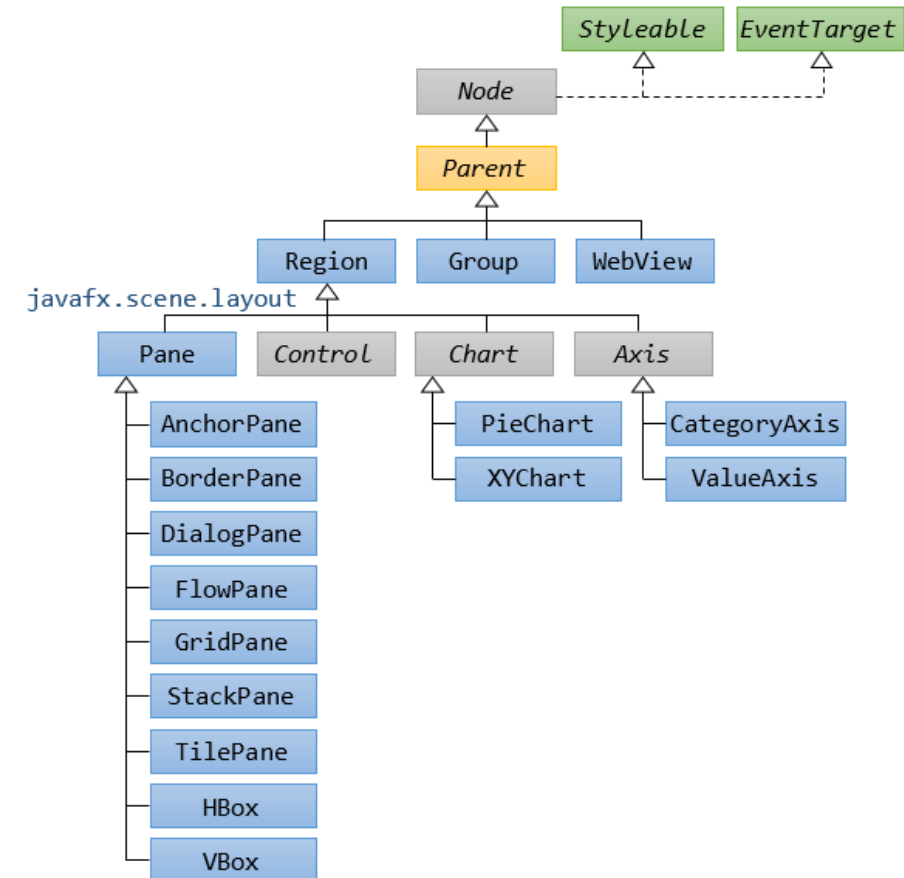
Layout Panes

- Kurzer Überblick über die folgenden Layout Panes:

- BorderPane
- HBox
- VBox
- GridPane
- FlowPane
- StackPane

- Weitere Layout Panes siehe

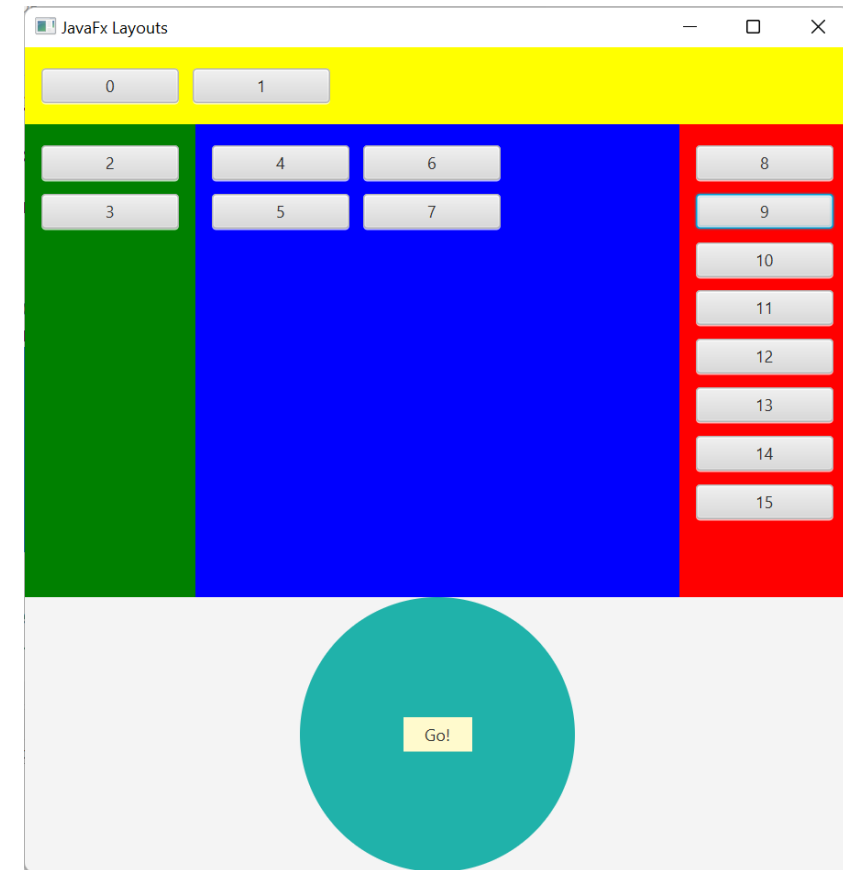
docs.oracle.com/javase/8/javafx/layout-tutorial/builtin_layouts.htm





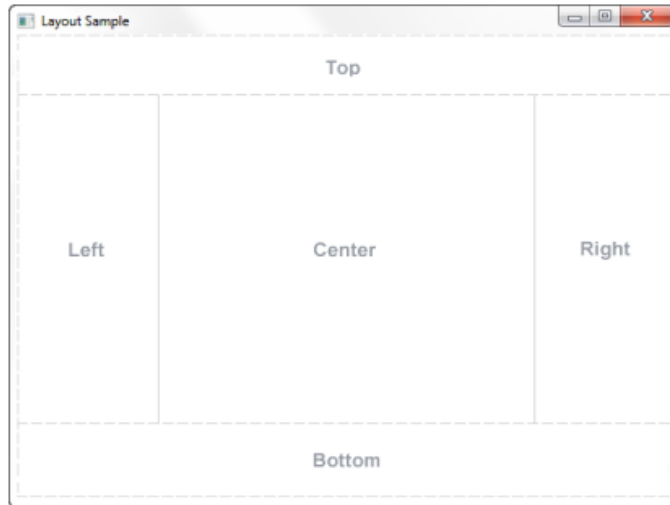
Layout Pane Beispiel

- Innerhalb des Beispiels werden größtenteils Schaltflächen (Buttons) angeordnet.
- Das Layout, das die Struktur bestimmt, ist ein `BorderPane`, in das die anderen Layouts integriert werden

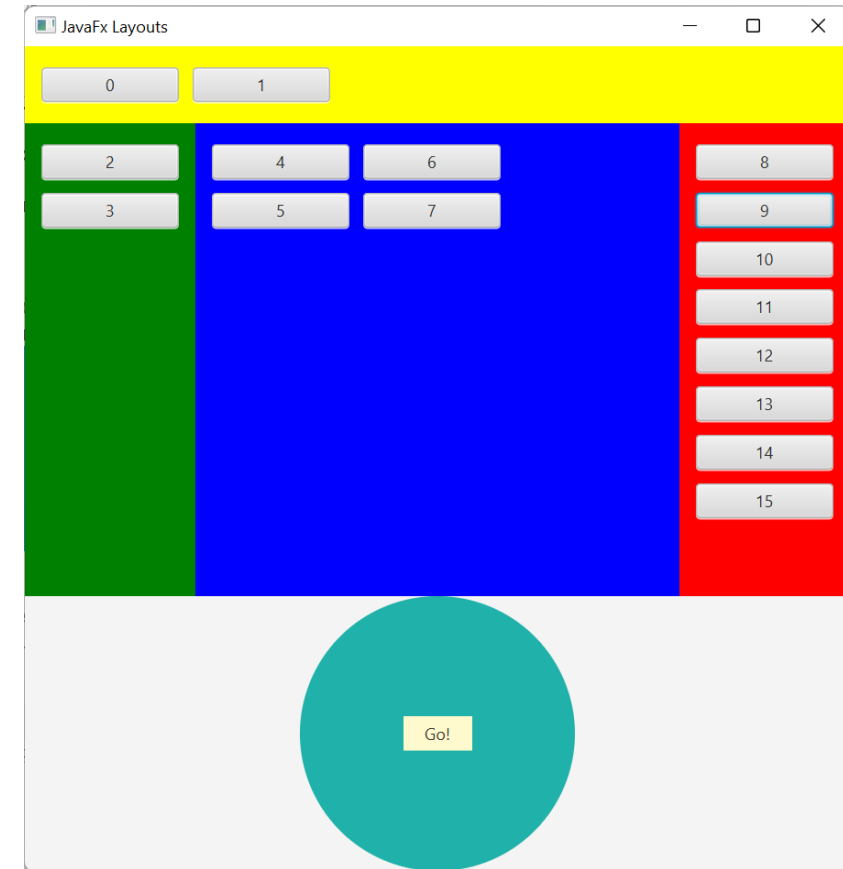




Layout Pane Beispiel



```
BorderPane pane = new BorderPane();  
pane.setTop(hbox);  
pane.setLeft(vbox);  
pane.setCenter(grid);  
pane.setRight(flow);  
pane.setBottom(stack);
```





Vorbereitung – Buttons erzeugen

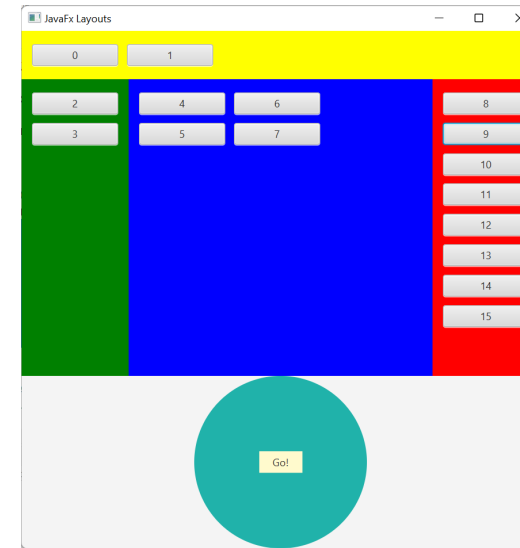
- Als Elemente der GUI verwenden wir eine Reihe von Buttons, die durchnummeriert werden.

```
Button[] buttonArr = new Button[16];  
for (int i = 0; i < buttonArr.length; i++) {  
    buttonArr[i] = new Button(i + "");  
    buttonArr[i].setPreferredSize(100, 20);  
}
```





Vbox und HBox



```
HBox hbox = new HBox();  
hbox.getChildren().addAll(buttonArr[0], buttonArr[1]);  
hbox.setStyle("-fx-background-color: yellow;");  
hbox.setPadding(new Insets(15,12,15,12)); // Abstände zum umgebenden Element  
hbox.setSpacing(10); // Abstand zwischen Knoten
```

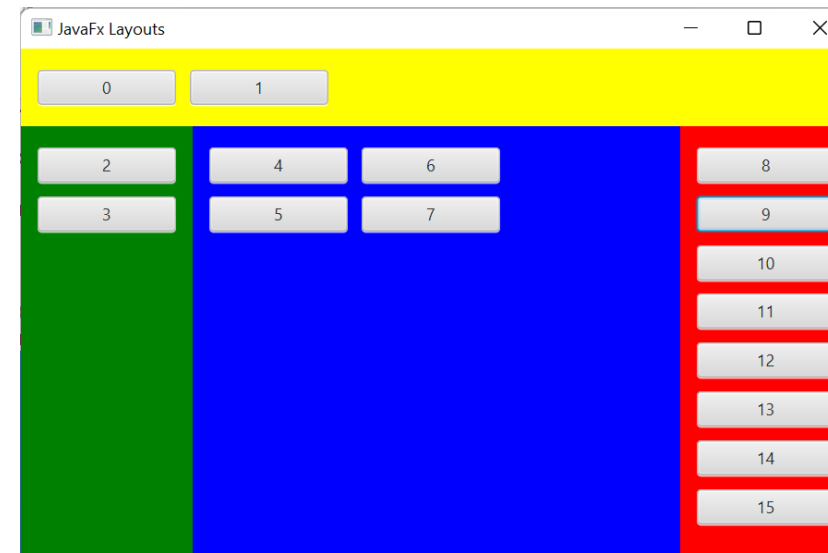
```
VBox vbox = new VBox();  
vbox.getChildren().addAll(buttonArr[2], buttonArr[3]);  
vbox.setStyle("-fx-background-color: green;");  
vbox.setPadding(new Insets(15,12,15,12));  
vbox.setSpacing(10);
```



GridPane



- Flexibler Layout-Container
- Im Grid-Layout werden die GUI-Komponenten in einem 2-dimensionalen Gitter angeordnet. Die Dimensionen des Grids werden automatisch abgeleitet.
- `add(n, spalte, zeile)`



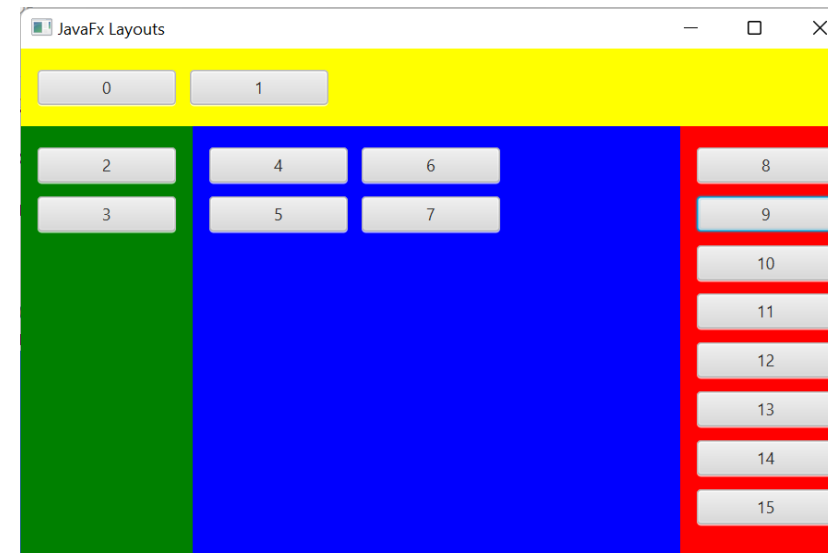
```
GridPane grid = new GridPane();  
grid.setHgap(10); // horizontaler Abstand der Spalten  
grid.setVgap(10); // vertikaler Abstand der Zeilen  
grid.setStyle("-fx-background-color: blue;");  
grid.setPadding(new Insets(15,12,15,12));  
grid.add(buttonArr[4], 0, 0);  
grid.add(buttonArr[5], 0, 1);  
grid.add(buttonArr[6], 1, 0);  
grid.add(buttonArr[7], 1, 1);
```



FlowPane



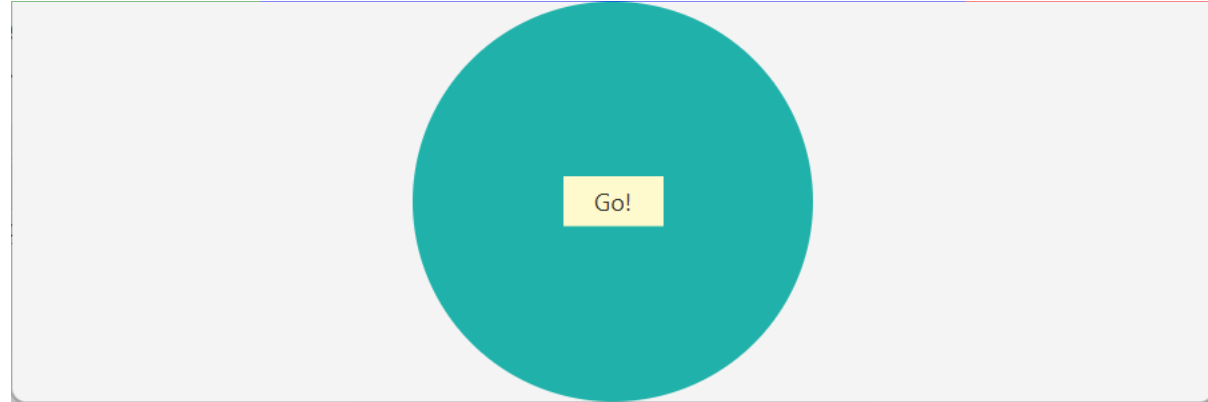
- Bei dem Layout FlowPane werden die Elemente – je nach Spezifikation – horizontal bzw. vertikal angeordnet.
- Wenn die Breite bzw. Höhe nicht reicht, wird dynamisch ein Umbruch erzeugt-



```
FlowPane flow = new FlowPane(Orientation.VERTICAL);
flow.setStyle("-fx-background-color: red;");
flow.setVgap(10); // vertikaler Space zwischen Knoten
flow.setHgap(10); //horizontaler Space zwischen Knoten
flow.setPadding(new Insets(15,12,15,12));
                        // Abstände zum umgebenden Element
for (int i = 8; i < 16; i++) {
    flow.getChildren().add(buttonArr[i]);
}
```

StackPane

- Bei dem Layout StackPane werden die Elemente überlagert („Stack“).
- Zur Demonstration werden graphische Elemente (Rectangle, Circle) und ein Label verwendet.

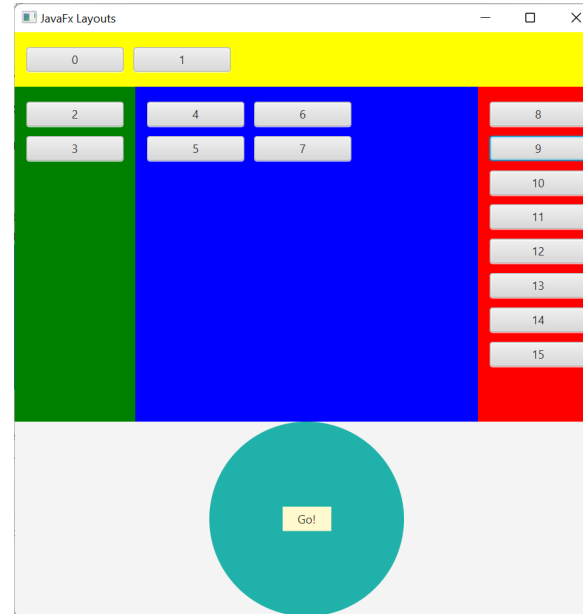


```
StackPane stack = new StackPane();  
stack.getChildren().addAll(new Circle(100, Color.LIGHTSEAGREEN),  
                             new Rectangle(50, 25, Color.LEMONCHIFFON),  
                             new Label("Go!"));
```





Vorbereitung – Aufbau von Stage und Scene



```
Scene scene = new Scene(pane, 600, 600);
```



```
primaryStage.setTitle("JavaFx Layouts");  
primaryStage.setScene(scene);  
primaryStage.show();
```



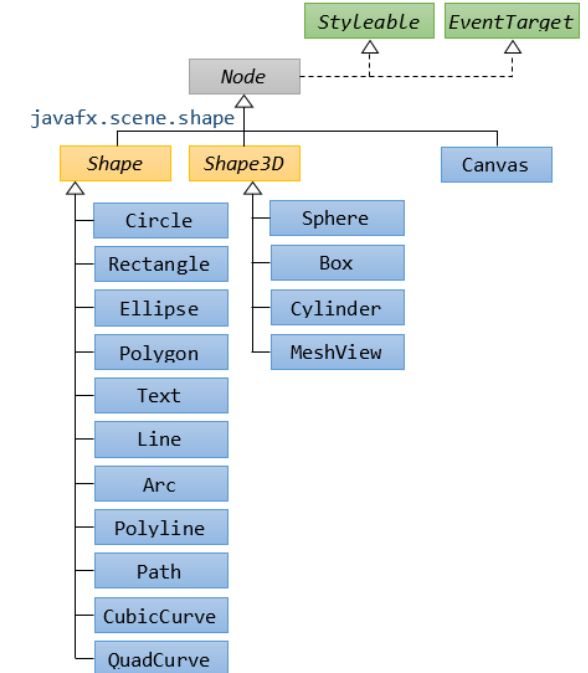
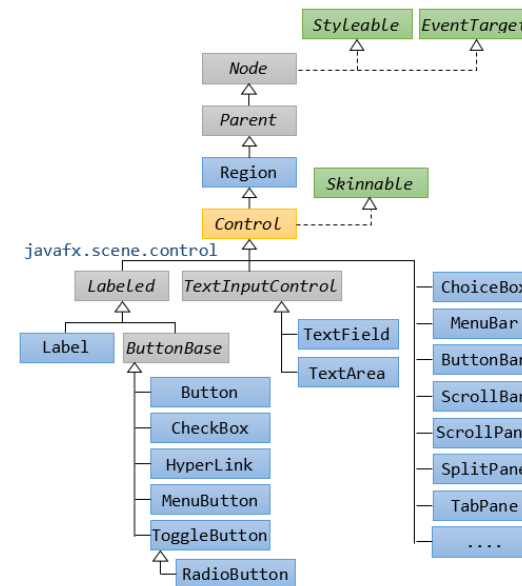
GUI Elemente

- In einem weiteren Beispiel werden GUI Elemente vorgestellt:

- Button (bereits verwendet)
- Label (bereits verwendet)
- CheckBox
- TextField
- MenuBar
- Circle (bereits verwendet)
- Rectangle (bereits verwendet)

- Weitere GUI Elemente siehe:

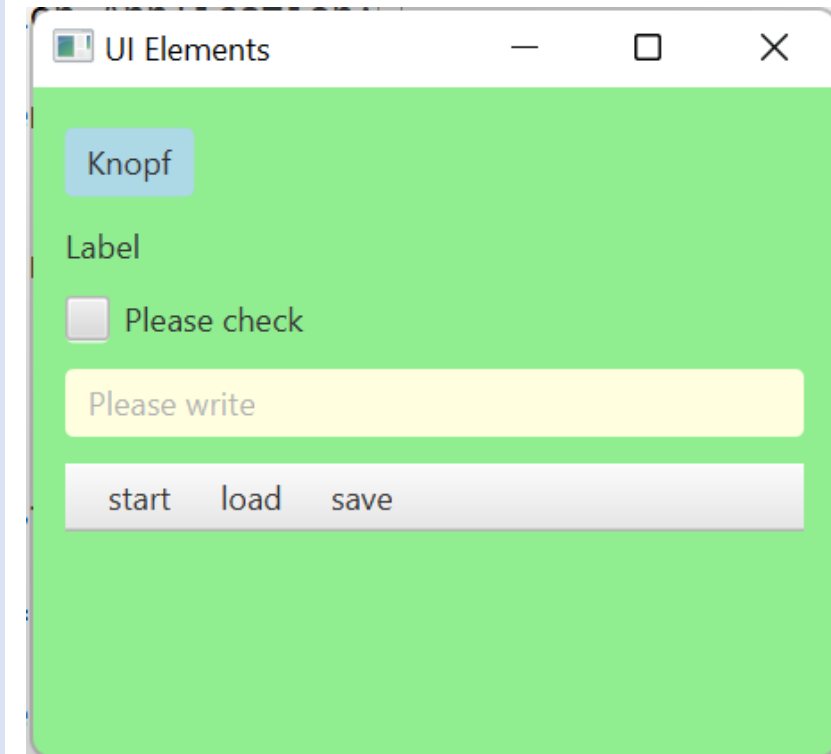
docs.oracle.com/javase/8/javafx/user-interface-tutorial/ui_controls.htm





Beispiel GUI Elemente

```
Button button = new Button();  
button.setText("Knopf");  
button.setStyle("-fx-background-color: lightblue;");  
  
Label label = new Label("Label");  
  
CheckBox check = new CheckBox("Please check");  
  
TextField textfield = new TextField();  
textfield.setPromptText("Please write");  
textfield.setStyle("-fx-background-color: lightyellow;");  
  
MenuBar menubar = new MenuBar();  
Menu start = new Menu("start");  
Menu load = new Menu("load");  
Menu save = new Menu("save");  
menubar.getMenus().addAll(start, load, save);
```



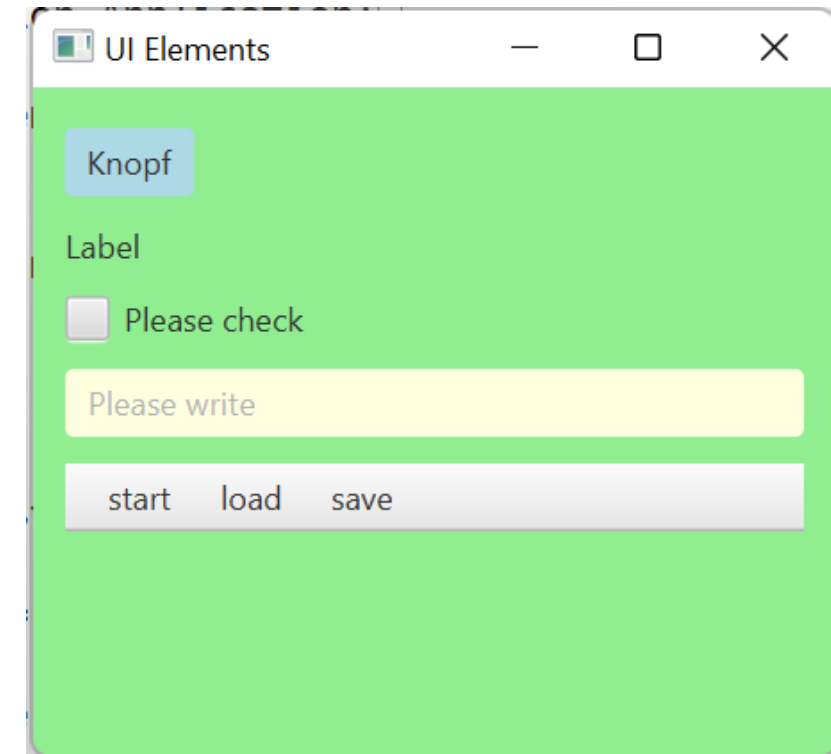


Beispiel GUI Elemente

```
VBox pane = new VBox();
pane.getChildren().addAll(button, label, check,
                           textfield, menubar);

pane.setStyle("-fx-background-color: lightgreen;");
pane.setPadding(new Insets(15,12,15,12));
pane.setSpacing(10);

Scene scene = new Scene(pane, 300,250);
primaryStage.setTitle("UI Elements");
primaryStage.setScene(scene);
primaryStage.show();
```





Wichtige Controls (teilweise verwendet)

- Beschriftungen (Label)
- Schaltflächen:
 - Button (einfache Schaltfläche)
 - RadioButton (für eine Auswahl aus mehreren Möglichkeiten)
 - ToggleButton (Schaltfläche, die zwei Zustände darstellen kann, z.B. an / aus)
- Auswahlboxen:
 - Checkbox (Auswahlbox mit 3 Zuständen: ausgewählt, nicht ausgewählt, undefiniert)
 - Choicebox (Aufklappliste, aus der genau ein Wert ausgewählt werden kann)
- Eingabefelder (TextField)
- Menüs
- Schieberegler (Slider)
- Fortschrittsbalken (ProgressBar) bzw. –indikator (ProgressIndicator)
- Button-Leiste (Toolbar)
- Scrollbarer Bereich (ScrollPane) bzw. Scrollbar (ScrollBar)
- Dialog für Datumsauswahl (DatePicker)
- Liste (ListView), Tabelle (TableView), Baumdarstellung (TreeView)



Programmieren

10. Graphische Benutzeroberflächen

10.3 Events und Event Handler



Wichtige Controls (teilweise verwendet)

- Wenn der Benutzer mit der GUI interagiert, lösen die GUI-Komponenten **Events** aus.
- Interagieren meint: Button drücken, Text eingeben, Checkbox auswählen, ...
- Informationen über Events werden in Event-Objekten verwaltet.
- Events sind in JavaFX Unterklassen-Objekte von `javafx.event.Event`.
- Jedes Event kapselt **Event-Typ**, **Quelle** (Source) und **Ziel** (Target).
 - Event-Typ: Erlaubt eine Ereignisunterscheidung, hat Name und Supertyp
 - Source: „Event-Ursprung“, Node, auf dem sich ein Event-Handler für das Event registriert hat; ändert sich, wenn ein Event mehrere Handler durchläuft
 - Target: Node, auf dem das Event stattgefunden hat; wird automatisch ermittelt. Das Target bleibt fest. Implementiert `javafx.event.EventTarget`.
- Für jede Event-Art gibt es in JavaFx eine Event-Klasse mit Event-Typen



Standard-Events in JavaFX (kleine Auswahl)

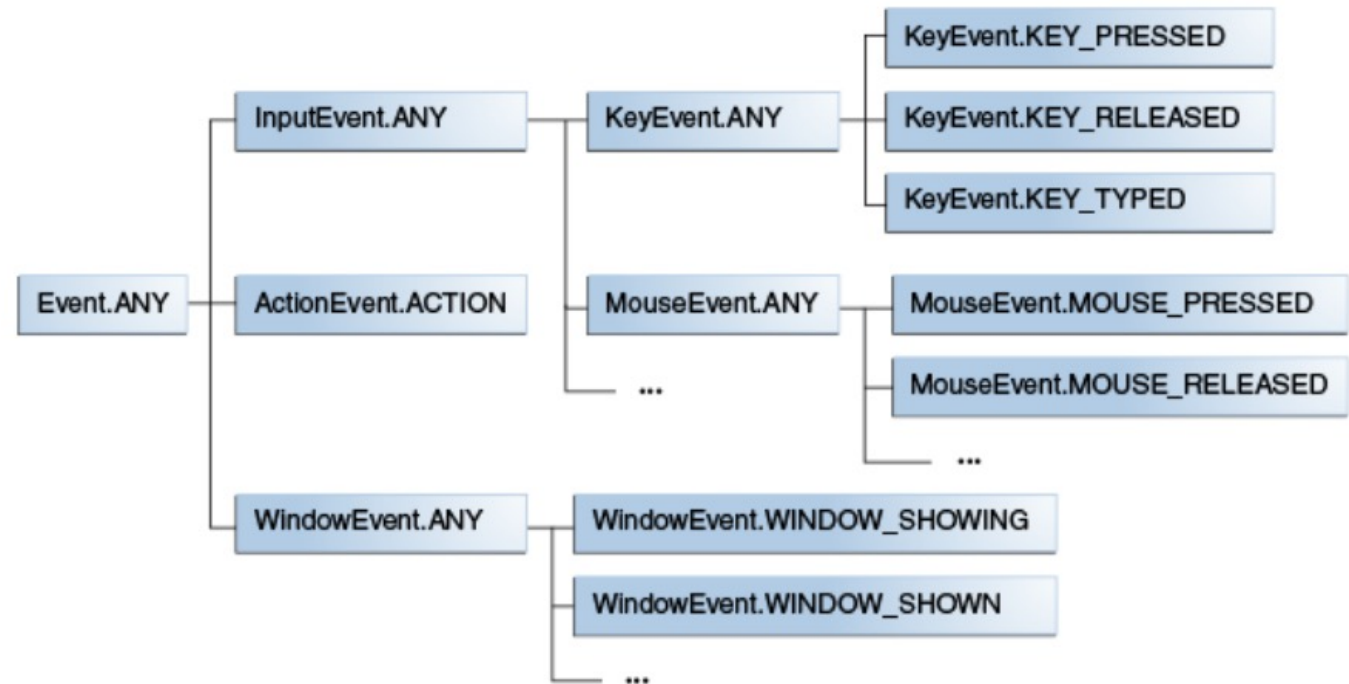
Event-Klasse	Auslöser	Event-Typ
ActionEvent	z.B. Klick auf Button	ACTION
KeyEvent	wenn Taste auf Tastatur gedrückt wird	CHAR_UNDEFINED KEY_PRESSED KEY_RELEASED KEY_TYPED
MouseEvent	Wenn Maus bewegt oder geklickt wird	MOUSE_CLICKED MOUSE_PRESSED MOUSE_RELEASED MOUSE_MOVED ...
WindowEvent	Wenn Fenster geöffnet oder geschlossen wird	WINDOW_CLOSE_REQUEST WINDOW_HIDDEN ...
ScrollEvent	Scrollbewegung (gestenbasiert, für Smartphone / Tablet)	SCROLL SCROLL_STARTED SCROLL_FINISHED



Hierarchie der Event-Typen

- Beispiel

- InputEvent hat Unterklassen KeyEvent, MouseEvent, Gestensteuerung etc.
- Eventtypen können speziell (z.B. KeyEvent.KEY_TYPED) oder als Obertypen (KeyEvent.ANY, InputEvent.ANY) betrachtet werden.



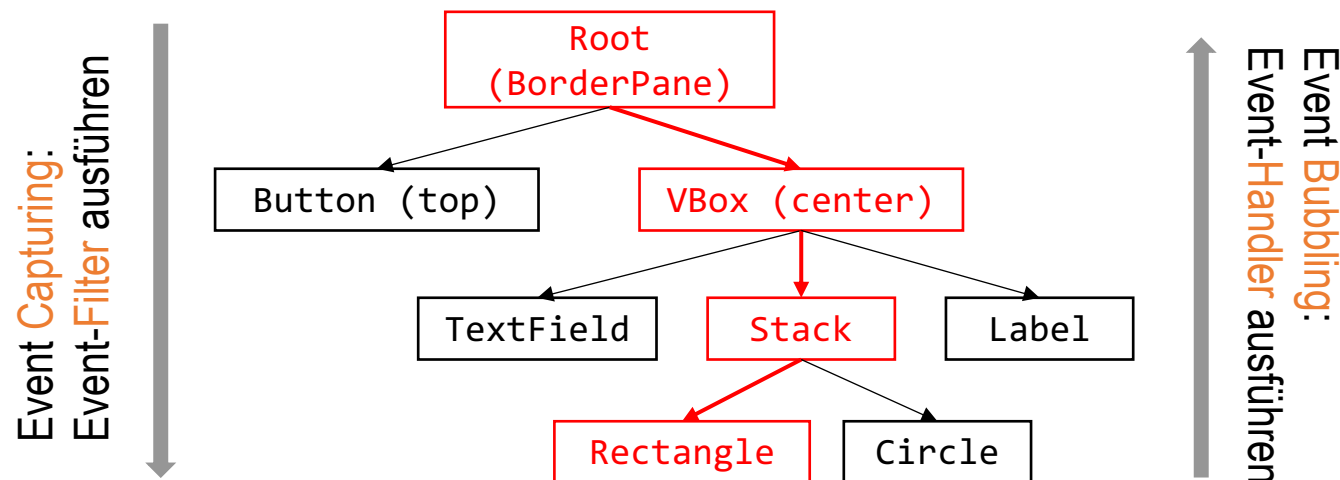
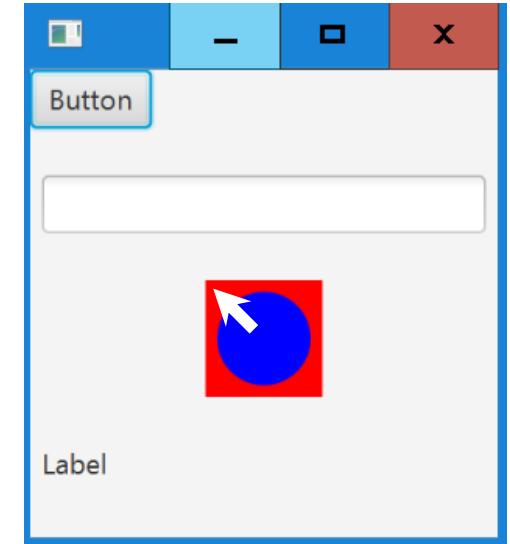


Event-Verarbeitung

- **Target Selection**
 - Wenn ein Event ausgelöst wird, wird der zugrundeliegende Node als „Target“ ermittelt.
 - Für jeden Eventtyp gibt es hierzu Regeln, siehe unten: docs.oracle.com
 - Bei MouseEvents z.B. ist das Ziel der oberste Knoten, über dem sich der Mauszeiger befindet, bei KeyEvents der Knoten, der den Eingabefokus hat.
- **Route Construction**
 - Ausgehend vom Root-Knoten des Scene-Graphen wird ein Pfad („event dispatch chain“) zum Target-Knoten bestimmt.
- **Event Capturing**
 - Das Event wird vom Root-Knoten über den ermittelten Pfad zum Target gesendet.
 - Wenn auf dem Pfad ein Event-Filter registriert wurde, wird dieser ausgeführt
 - bei Event-Konsumption wird das Ziel nicht erreicht.
 - **Event-Filter:** Ermöglicht, ein Event abzufangen oder die Event-Antwort zu überschreiben
- **Event Bubbling**
 - Nach Erreichen des Ziels reist das Event dieselbe Route vom Target zurück zur Wurzel des Scene-Graphen.
 - Entlang des Pfads werden alle registrierten Event-Handler aufgerufen, die auf das Event warten;
 - wenn das Event konsumiert wird, ist die Route beendet.
 - **Event-Handler:** Ermöglicht, die Reaktion auf ein Event festzulegen (Anwendungslogik)

Event-Verarbeitung: Beispiel

- Mauszeiger auf Rechteck platziert
 - Target: Rectangle
- EventHandler auf Stack registriert
- Event dispatch chain: roter Pfad im Scene-Graph
- Event source: Zuletzt durchlaufener Event-Handler





Möglichkeiten zur Event-Verarbeitung

- Drei alternative Möglichkeiten:
 1. Verwendung der von einem Node angebotenen „Convenience methods“ (*setOnEvent-Typ Methode*)
 - Vorteil: Sehr einfach und intuitiv
 - Nachteil: Für jedes Ereignis kann nur ein EventHandler registriert werden
 2. Generische Methoden zum Registrieren von EventHandlern (`addEventHandler`, `removeEventHandler`; entsprechend für Event-Filter)
 - Vorteil: Es können beliebig viele Handler für ein Event registriert werden
 - Nachteil: Evt. nicht ganz so intuitiv
 3. Verwendung eines eigenen EventDispatchers
 - Vorteil: Sehr mächtig
 - Nachteil: Implementierungsaufwand (Methode `dispatchEvent` implementieren)
 - Hier nicht behandelt!
- Nötig ist natürlich für die jeweilige Anwendung die Auswahl des / der Knoten, auf dem / denen ein (oder mehrere) Handler registriert werden sollen!



1. „Convenience methods“ für Event-Handler

- Einfache Form, um Event-Handler an Knoten zu registrieren: über „convenience methods“; Ausführung in der Event-Bubbling Phase.
- Events werden mit einer `setOnEvent`-Typ Methode an einem Node registriert.
- Bsp.: `setOnAction`, `setOnMouseClicked` ...
- Dazu muss ein Event-Handler-Objekt erzeugt werden, dass das Interface `EventHandler<T extends Event>` implementiert. Hierzu muss die Methode `handle` implementiert werden, die das Event als Parameter erhält.
- Übersicht:
https://docs.oracle.com/javase/8/javafx/events-tutorial/convenience_methods.htm



1. „Convenience methods“ für Event-Handler: Beispiel 1

- Da die als Event-Handler realisierten Klassen meist relativ klein bleiben, können sie auch als anonyme innere Klasse realisiert werden.

```
Button button = new Button("Hello World");

button.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent arg0) {
        System.out.println("Hello World Button pressed");
    }
});

StackPane root = new StackPane();
root.getChildren().add(button);
```





2. Generische Methoden für Event-Handler

- Neben der *setOnEvent-Typ* Methode können Event-Handler und Event-Filter auch über die folgenden generischen Methoden realisiert werden:
 - `addEventFilter`, `removeEventFilter`
<http://docs.oracle.com/javase/8/javafx/events-tutorial/filters.htm>
 - `addEventHandler`, `removeEventHandler`
<http://docs.oracle.com/javase/8/javafx/events-tutorial/handlers.htm>
 - Falls mehrere EventHandler für ein Event registriert sind, wird der Handler, der über *setOnEvent-Typ* gesetzt wurde, als letzter Handler nach den über `addEventHandler` registrierten ausgeführt.
 - Spezifischere Handler / Filter werden vor generischen aufgerufen.
- siehe Dokumentation



2. Generische Methoden für Event-Handler Beispiel



```
Button button = new Button("Hello World");

button.addEventHandler(ActionEvent.ACTION, new EventHandler<ActionEvent>() {

    @Override
    public void handle(ActionEvent arg0) {
        System.out.println("Hello World Button pressed (handler)");
    }
});

StackPane root = new StackPane();
root.getChildren().add(button);
```



Programmieren

10. Graphische Benutzeroberflächen

10.4 Design Pattern für eine Anwendung mit JavaFX



Entwurf einer Anwendung mit GUI

- Technische Möglichkeiten von JavaFX
 - Darstellung: JavaFX-GUI-Elemente
 - Ereignisse: Events
 - Auslösen von Reaktionen: Event Handler
- Fragestellung:
 - Ablaufsteuerung und Verbindung zwischen JavaFX-Anzeige und Daten im Anwendungsmodell
 - Viele Möglichkeiten der Implementierung, aber: Ohne Entwurfsplan endet das in sehr unübersichtlichen Programmstrukturen!
- Lösung:
 - Praktisch alle Anwendungen mit graphischer Benutzeroberfläche werden heute nach dem **Model-View-Controller-Pattern** (oder einer Variante: Model-View-Presenter, Model-View-ViewModel) aufgebaut.



Ziele des objekt-orientierten Entwurfs

- Korrektheit
 - Entwurf soll Implementierungsfehler vermeiden helfen.
- Verständlichkeit
 - Code soll einfach und verständlich sein.
- Wiederverwendung
 - Erstellte Lösungen sollen in ähnlichen Kontexten erneut verwendbar sein.
- Effizienz
 - Entwurf soll in Bezug auf Laufzeit und Speicher eine effiziente (und arbeitsteilige) Implementierung unterstützen.



Entwurfsmuster - Design Pattern

- Ein **Design Pattern (Entwurfsmuster)** beschreibt einen bewährten Ansatz zur Lösung einer Software-Entwurfsaufgabe, die wiederholt und in verschiedenen Kontexten auftritt.
- Entwurfsmuster
 - geben häufig verwendeten Lösungen einen Namen und erleichtern damit die Kommunikation
 - beschreiben Details einer Lösung
 - dokumentieren das Erfahrungswissen zu den Vor- und Nachteilen eines Lösungsansatzes
- Bekannteste Quelle
 - Design Patterns - Elements of Reusable Object-Oriented Software von Erich Gamma, Richard Helm, Ralph Johnson und John Vlissides, 1994
 - bekannt als GoF = Gang of Four - die „Viererbande“, Spitznamen der Autoren

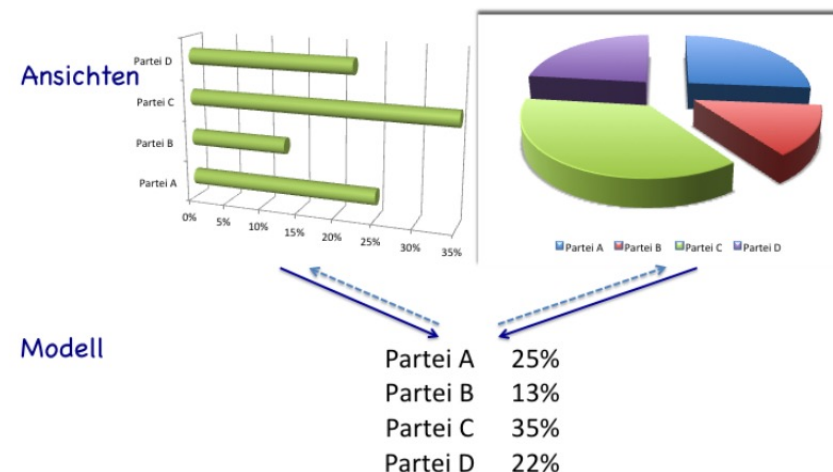


Beispiel: Architekturprinzip

- **Single Responsibility Principle:** Jede Entität im Entwurf hat eine einzige Aufgabe.
- Zitate:
 - “There should never be more than one reason for a class to change.” R.C. Martin, 1997
 - “A functional unit on a given level of abstraction should only be responsible for a single aspect of a system’s requirements. An aspect of requirements is a trait or property of requirements, which can change independently of other aspects.” Ralph Westphal, 2012
- Beispiel: Software für Einkommenssteuer
 - Änderungsgründe:
 - Änderung in der Steuergesetzgebung führt zu neuer Formel
 - Font in der Bildschirmausgabe soll geändert werden
 - Fazit: Die Anpassung der Software an die verschiedenen Änderungen sollte in verschiedenen Klassen durchzuführen sein.

Beispiel Observer Pattern („Beobachter-Muster“)

- Aufgabe
 - Es gibt ein Objekt, dessen Zustandsänderungen für mehrere andere Objekte von Bedeutung sind, da diese im Falle einer Änderung ebenfalls Aktualisierungen vornehmen müssen. Die Objekte sollen aber möglichst unabhängig voneinander entwickelt werden.
- Beispiel
 - Dieselben Werte und Wertänderungen werden gleichzeitig in einem Balkendiagramm, einem Kreisdiagramm etc. dargestellt.



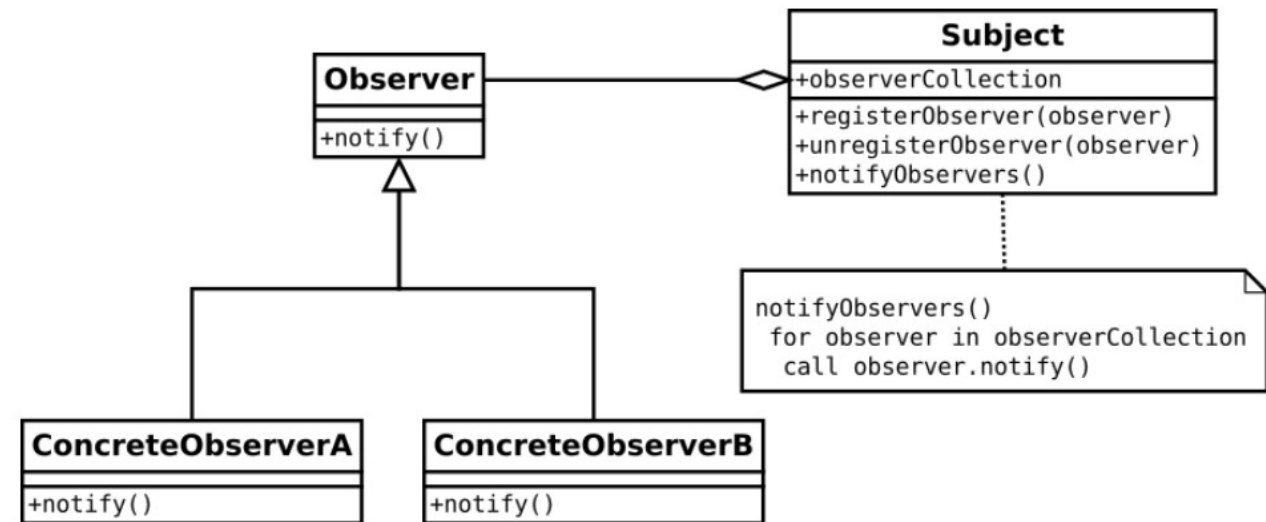


Observer Pattern in der GUI-Entwicklung

- Single Responsibility Principle
 - Trennung der Verantwortlichkeit
- Verantwortlichkeiten
 - Modell: Fachliche Klassen: Daten und Logik („Subjekt“, „Observable“)
 - Ansicht: Darstellung der Daten („Observer“)
- Abhängigkeiten
 - Ansicht muss auf das Modell zugreifen.
 - Modell kann unabhängig von der Ansicht entwickelt werden.
 - Verschiedene GUIs können auf demselben Modell arbeiten und auch verschiedene Aspekte darstellen.

Observer Pattern

- Subjekt bietet Abonentendienst an, bei dem sich Beobachter an- und abmelden können (`registerObserver(...)`, `unregisterObserver(...)`).
- Das Subjekt schickt bei einer Zustandsänderung Benachrichtigungen (`notify()`) an alle Abonnenten.
- Subjekt: Observable, deren Zustandsänderungen beobachtet werden sollen.
- Observer: Beobachter, der über Änderungen informiert werden möchte.



Quelle: "Observer" by WikiSolved - Own work. Licensed under Public Domain via Wikimedia Commons -



Observer Pattern in Java

- Java stellt Klasse `Observable` in `java.util` bereit, in der die Methoden des Subjekts bereits implementiert sind
 - Mit `notifyObservers(...)` werden alle Observer von Zustandsänderungen des Subjekts informiert, indem das Subjekt die Liste seiner Observer durchläuft und bei allen die `update()`-Methode aufruft.
- ... und ein Interface `Observer`, das die Implementierung der Methode `update(...)` verlangt, die für jeden Observer die Reaktion auf eine Aktualisierung des Subjekts beschreibt.
- Seit Java 9 wird von der Nutzung von `java.util.Observable` und `Observer` abgeraten: `deprecated`
- Alternativen in Java
 - Java Beans Property + Listener (siehe später)
 - Concurrent data structures in `java.util.concurrent` – nicht behandelt
 - Flow API (Publish / Subscribe) – nicht behandelt



Beispiel: Observer Pattern in Java (selbst gemacht)

- Beispiel: Wetterdaten werden in einer Wetterstation erfasst und sollen bei Änderung an verschiedene Stellen angezeigt werden (als Text oder in einer GUI)
- Das Subject Interface kann allgemeingültig sein
- Das Observer Interface beinhaltet normalerweise schon Informationen darüber, welche Daten aktualisiert werden (hier Temperatur, Luftfeuchtigkeit und Druck).

```
public interface Subject {  
    public void registerObserver(Observer o);  
    public void removeObserver(Observer o);  
    public void notifyObserver();  
}
```



```
public interface Observer {  
    public void update(float temp, float humidity, float pressure);  
}
```





Beispiel: Observer Pattern in Java (selbst gemacht)

```
public class WeatherData implements Subject {  
  
    private List<Observer> observers;  
    private float pressure;  
    private float temperature;  
    private float humidity;  
  
    public WeatherData() {  
        observers = new ArrayList<>();  
    }  
  
    @Override  
    public void registerObserver(Observer o) {  
        observers.add(o);  
    }  
  
    @Override  
    public void removeObserver(Observer o) {  
        observers.remove(o);  
    }  
}
```



```
@Override  
public void notifyObserver() {  
    for (Observer obs : observers) {  
        obs.update(temperature, humidity, pressure);  
    }  
}  
  
public void setMeasurements(float tmp, float hum,  
                             float pres) {  
  
    this.temperature = tmp;  
    this.humidity = hum;  
    this.pressure = pres;  
    notifyObserver();  
}  
}
```





Beispiel: Observer Pattern in Java (selbst gemacht)

```
public interface DisplayElement {  
    public void display();  
}
```



```
public class TextWeatherDisplay implements Observer,  
                                                DisplayElement {  
  
    private float temperature, humidity, pressure;  
  
    public TextWeatherDisplay(Subject weatherdata) {  
        weatherdata.registerObserver(this);  
    }  
  
    @Override  
    public void display() {  
        System.out.println("Current conditions:" + temperature +  
            "°C, " + humidity + "% humidity and " + pressure + " hpa");  
    }  
  
    @Override  
    public void update(float temp, float humidity, float pressure) {  
        this.temperature = temp; this.humidity = humidity;  
        this.pressure = pressure;  
        display();  
    }  
}
```



```
public class GUIWeatherDisplay implements Observer, DisplayElement {  
    Label label = new Label("Weather Observations:");  
    private float temperature, humidity;  
    private Stage stage;  
  
    public GUIWeatherDisplay(WeatherData data, Stage stage) {  
        data.registerObserver(this);  
        this.stage = stage;  
        try { setup(); } catch (Exception e) {e.printStackTrace();}  
        stage.show();  
    }  
  
    public void display() {  
        String s = label.getText();  
        s = s + System.lineSeparator() + "Current conditions:" +  
            temperature + "°C and " + humidity + "% humidity";  
        label.setText(s);  
    }  
  
    public void update(float temp, float humidity, float pressure) {  
        this.temperature = temp; this.humidity = humidity;  
        display();  
    }  
  
    public void setup() throws Exception {  
        VBox pane = new VBox();  
        pane.getChildren().addAll(label);  
        pane.setStyle("-fx-background-color: lightgreen;");  
        pane.setPadding(new Insets(15,12,15,12)); pane.setSpacing(10);  
        Scene scene = new Scene(pane, 300,250);  
        stage.setTitle("UI Elements");  
        stage.setScene(scene);  
    }  
}
```





Beispiel: Observer Pattern in Java (selbst gemacht)

```
public class WeatherStation extends Application{   
  
    public static void main(String[] args) {  
        launch(args);  
    }  
  
    @Override  
    public void start(Stage primaryStage) {  
        WeatherData data = new WeatherData();  
        new TextWeatherDisplay(data);  
        new GUIWeatherDisplay(data, primaryStage);  
        for (int i = 0; i < 10; i++) {  
            delay(1000*i, data,  
                20+i, 50*i%100, 1025+2*i);  
        }  
    }  
}
```

```
public void delay(long m, WeatherData data,   
                float f1, float f2, float f3) {  
    Task<Void> sleeper = new Task<Void>() {  
        @Override  
        protected Void call() throws Exception {  
            try {  
                Thread.sleep(m);  
            } catch (InterruptedException e) {  
            }  
            return null;  
        }  
    };  
    sleeper.setOnSucceeded(new  
        EventHandler<WorkerStateEvent>() {  
            @Override  
            public void handle(WorkerStateEvent event) {  
                data.setMeasurements(f1, f2, f3);  
            }  
        }  
    ));  
    new Thread(sleeper).start();  
}
```



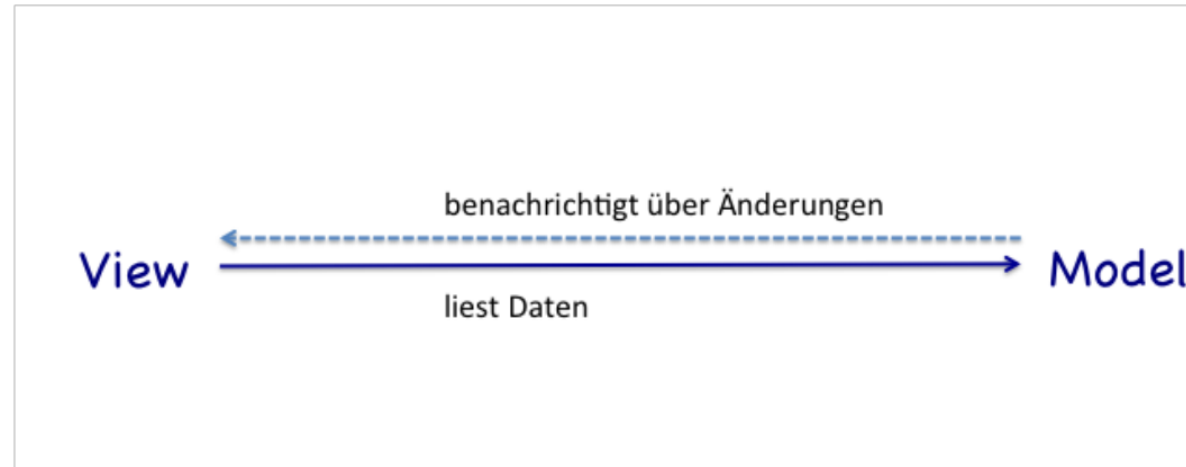
Architektur-Muster: Model-View / Model-View-Controller

- Grundidee: Aufteilung des Codes in einen
 - allgemeinen, oberflächen-unabhängigen Teil – das **Model**
 - und einen oberflächenspezifischen Code – der **View**
- Der oberflächenspezifische Code kann wiederum aufgeteilt werden in
 - Code zur reinen Anzeige – der View
 - Code zur Verarbeitung von Benutzereingaben und Steuerung der Oberfläche – der **Controller**
- Aber:
 - Kein einheitliches Verständnis über die konkrete Umsetzung dieser Aufteilung, die Aufgaben der einzelnen Komponenten sowie die Beziehungen / Sichtbarkeiten
 - U.a. auch deswegen, weil die Idee von MVC (und seinen Varianten) auf verschiedenen Abstraktionsniveaus anwendbar ist
 - Einzelnes UI-Control, z.B. Textfeld-Komponente eines UI-Toolkits
 - Gesamt- oder Teil-Ansichten der Anwendung, z.B. ein Formular
 - Gesamtarchitektur einer Anwendung
- Im Folgenden: Vorstellung von Model-View-Controller und Varianten



Model-View: Abhängigkeiten

- View zeigt im Allgemeinen nur eine geeignete Sicht, eine Auswahl der Daten des Modells an.
- View agiert als Observer des Modells, um stets den aktuellen Stand darstellen zu können.



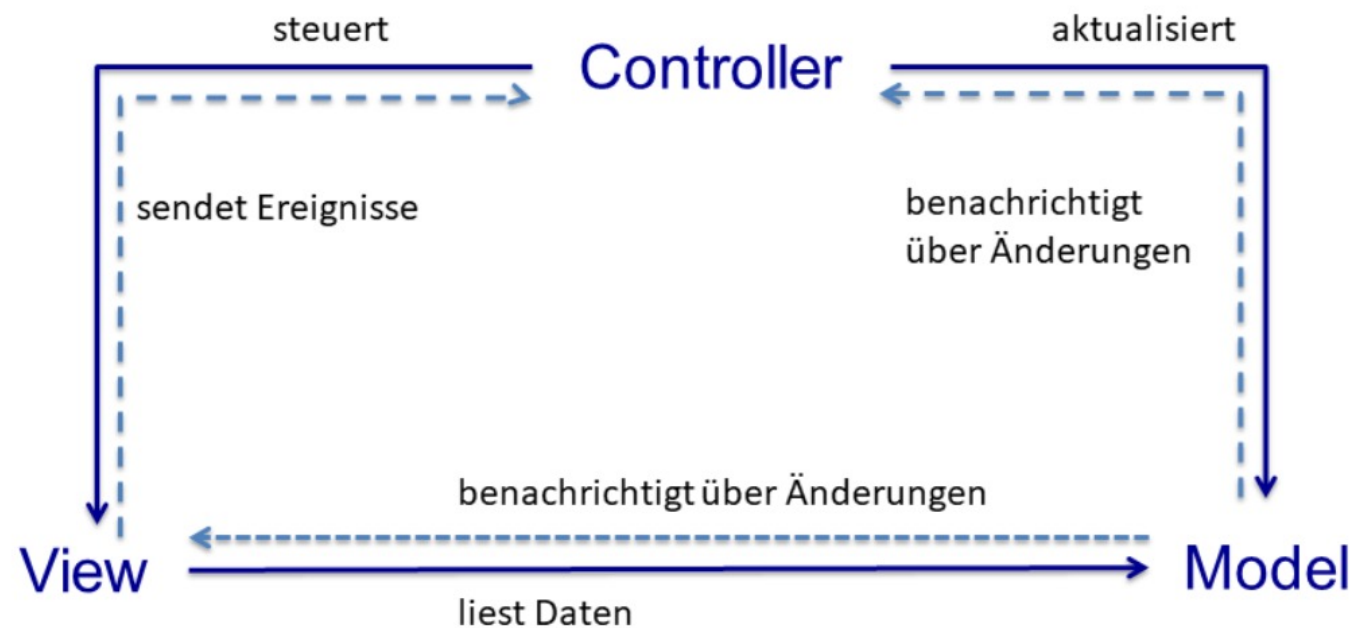


Model-View-Controller (MVC)

- **Model-View-Controller** ist ein **Entwurfsmuster** zur Implementierung von Benutzeroberflächen.
 - schon seit vielen Jahren als Entwurfsmuster in Verwendung
 - in den verschiedensten Programmierumgebungen genutzt und fester Architekturbestandteil in GUI-Frameworks.
- **Verantwortlichkeiten:**
 - **Model:** Fachliche Klassen, Daten und Fach-Logik
 - **View:** Darstellung von Daten, Style, Layout, initialisiert, organisiert und speichert GUI-Elemente
 - **Controller:** verbindet Model und View, Steuerung und Abfolge von Views, nimmt Benutzereingaben an und leitet sie ans Modell

Model-View-Controller: Abhängigkeiten

- Benutzereingaben im View lösen Ereignisse aus, die der Controller behandelt.
- Controller veranlasst Änderungen am Modell gemäß der Benutzereingaben.
- Controller implementiert die Ablauflogik der Benutzerschnittstelle.
- Controller agiert als Observer des Modells.





Model-View-Controller: View

- Ist für die Präsentation von Daten zuständig
- Initialisiert, organisiert und speichert alle Oberflächenelemente
- Ist für Style und Layout zuständig
- „Kennt“ das Modell
- Wird vom Modell über Änderungen der Daten informiert
- Aktualisiert die Anzeige, wenn es benachrichtigt wird, dass es im Modell Änderungen gibt.

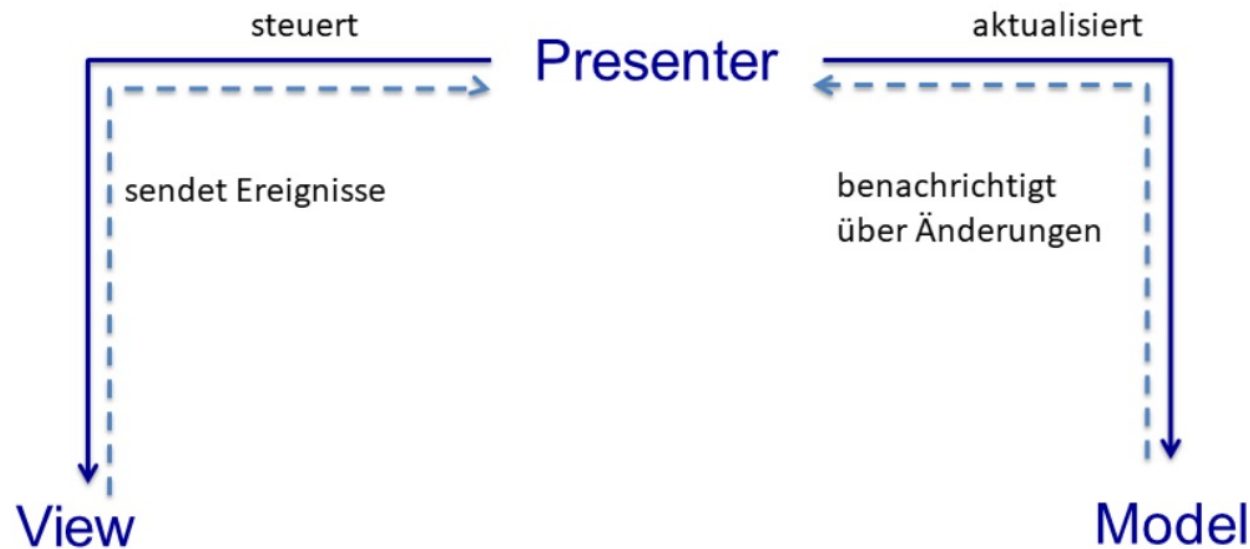


Model-View-Controller: Controller

- Kennt das Modell und die View
- Steuert die View in allem, was über Datenaktualisierung hinaus geht
- Behandelt Ereignisse, die der Benutzer auslöst
- Setzt die Eingaben des Benutzers in Methodenaufrufe des Modells um
- Faustregel: Der Controller sollte so geschrieben sein, dass ein Entwickler das Verhalten der Benutzerschnittstelle daraus erfassen kann.
- Beachte: Controller und View sind eng gekoppelt; oft wird jedem View ein Controller zugeordnet.
- Falls nur künstliche Trennung zwischen Controller und View möglich: View und Controller zusammenfassen: nameVC-Klasse.

Variante 1: Model-View-Presenter

- Bei MVC kennt die View das Modell.
- Variante: Stärkere Trennung von View und Model: die View wird nur über den Presenter aktualisiert: **Model-View-Presenter**





Variante 2: Model-View-ViewModel

- MVVM: JavaFX unterstützt die **Model-View-ViewModel** Variante.





Variante 2: Model-View-ViewModel

- View:
 - Enthält die Schaltflächen (Buttons, Textfelder usw.)
- ViewModel:
 - Gibt die aktuellen Werte von Textfeldern, Tabellen usw. und aktuelle Informationen wie „Button X ist inaktiv“ nach außen
 - Implementierung der Präsentationslogik:
 - Was passiert, wenn ein Button geklickt wird?
 - Wie setzt sich der Text eines Labels zusammen?
 - Unter welchen Bedingungen ist Button X ausgegraut? etc.
- View und ViewModel müssen stets synchron gehalten werden:
 - Wenn der UI-Zustand vom Nutzer verändert wird (z.B. Eingabe in Textfeld), reicht die View diese Änderungen umgehend an das ViewModel weiter
 - Andersherum werden auch Änderungen, die vom ViewModel ausgehen, umgehend von der View angezeigt
 - Synchronisation über Properties und Databinding (siehe nächster Abschnitt)
- Vorteile dieser Aufteilung:
 - Unabhängige Entwicklung von View (ohne Logik) und ViewModel (ohne Design)
 - Vereinfachtes Testen (ViewModel ohne UI-Controls, View nur wenig testbedürftig)



Zusammenfassung Model-View-Controller

- Idee: Trennung von fachlichem Modell und Anzeige einer Anwendung
- Faustregel: Keine JavaFX-Importe im Modell
- MVC kommt heute in praktisch allen komplexen Anwendungen mit GUI oft auch auf verschiedenen Ebenen vor



Model-View-Controller: Offene Fragen

- Idee: Trennung von fachlichem Modell und Anzeige einer Anwendung.
- Es gibt aber Aufgaben, deren Zuordnung nicht so eindeutig ist.
 - Fachlogik: Historisch oft noch im Controller, bei modernen objekt-orientierten Ansätzen im Modell
 - Validierung von Benutzereingaben: Einfache Formatprüfungen oft im View, tieferliegende Konsistenzprüfungen im Modell
 - Daten-Formatierung und Internationalisierung: erfordert oft Modellverständnis und wird daher bisweilen dem Modell zugeordnet, ist aber eine Darstellungsaufgabe. Wird es im View behandelt, wird dieser aufgebläht.



Programmieren

10. Graphische Benutzeroberflächen

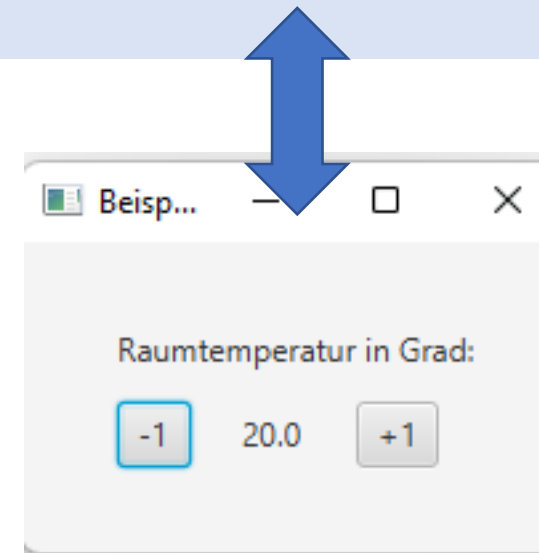
10.5 Properties, Listener
und Bindungen



Properties: Motivation

- Design Pattern: Aufteilung des Codes in Modell / Daten und Darstellung / GUI
- Frage: Wie wird die Darstellung mit dem Modell synchron gehalten?
- Aktuelle Raumtemperatur (Daten) soll in der GUI angezeigt werden
- Änderungen der Raumtemperatur über die GUI sollen an die Daten weitergereicht werden

```
public class Raum {  
    private double temp;  
  
    public double getTemp() {return temp; }  
  
    public void setTemp(double temp) {  
        this.temp = temp;  
    }  
}
```





Properties definieren

- Properties: Wrapper-Klassen um normale Java-Datentypen
 - IntegerProperty, StringProperty, BooleanProperty, generische ObjectProperty
 - Erlauben Observer-Mechanismus über die Registrierung von Listnern, mit denen eine Reaktion auf eine Wertänderung einer Property festgelegt werden kann
 - Properties ermöglichen weiterhin eine Synchronisation über Bindungen
 - Properties können auch Events aussenden, wenn die Werte sich ändern, und man kann Event-Handler mit diesen Events verknüpfen (onActionProperty)



Properties der JavaFX-Controls

- Alle JavaFX-Controls bieten fast alle ihrer Eigenschaften als Properties an
- Properties von `TextField`, `Label` und co.
 - `textProperty(): StringProperty`
 - `disableProperty(): BooleanProperty`
 - `rotateProperty(): DoubleProperty`
 - ...



Properties definieren

- Ersetzung primitiver Datentyp durch zugehörige Property
- get- und set-Methode für den gekapselten Wert / Objekt definieren
- get-Methode für die Property selbst definieren (Namenskonvention)

```
public class Raum {  
    private DoubleProperty temp = new SimpleDoubleProperty();  
    public double getTemp() { return temp.get(); }  
    public void setTemp(double temp) { this.temp.set(temp); }  
    public DoubleProperty tempProperty() { return this.temp; }  
}
```





Listener

- An Properties lassen sich Listener registrieren (z.B. ChangeListener)
- Lösen Reaktionen auf Änderungen des Wertes / Objektes aus (Observer-Prinzip)
- Listener lassen sich an alle Observable registrieren (Properties sind Observable)
- Anwendung des Konzeptes der anonymen inneren Klasse (später: Lambda-Ausdruck)

```
raum.tempProperty().addListener(new ChangeListener<Object>() {  
    @Override  
    public void changed(ObservableValue<?> wert, Object alt, Object neu) {  
        // reagiert auf Temperaturänderung  
    }  
});
```





EventHandler vs. Listener

- **EventHandler:**
 - Wird an einen Node registriert
 - Implementiert `handle`
 - Reaktion auf Ereignis
- **(Change)Listener:**
 - Wird an eine `ObservableValue`, z.B. eine Property des Nodes, registriert
 - Implementiert `changed`
 - Reaktion auf Wertänderung eines Observable

```
node.setOnAction(new EventHandler<ActionEvent>() {   
    @Override  
    public void handle(ActionEvent event) {  
        // ...  
    }  
});
```

```
prop.addListener(new ChangeListener<Object>() {   
    @Override  
    public void changed(ObservableValue<?> wert,  
                        Object alt, Object neu) {  
        // ...  
    }  
});
```



Properties: Beispiel

```
public class Raum {  
    private DoubleProperty temp = new SimpleDoubleProperty();  
    public double getTemp() { return temp.get(); }  
    public void setTemp(double temp) { this.temp.set(temp); }  
    public DoubleProperty tempProperty() { return this.temp; }  
}
```





Properties: Beispiel

```
public class View {  
    private Stage stage;  
    private Scene scene;  
    Button buttonPlus, buttonMinus;  
    Label tempLabel, titelLabel;  
  
    public View(Stage stage) {  
        this.stage = stage;  
        titelLabel = new Label("Tmp in Grad:");  
        tempLabel = new Label();  
        tempLabel.setAlignment(Pos.CENTER);  
        buttonPlus = new Button("+1");  
        buttonMinus = new Button("-1");  
  
        GridPane grid = new GridPane();  
        grid.setHgap(20);  
        grid.setVgap(10);
```



```
        grid.setAlignment(Pos.CENTER);  
        grid.add(titelLabel, 0, 0, 3, 1);  
        grid.add(buttonMinus, 0, 1);  
        grid.add(tempLabel, 1, 1);  
        grid.add(buttonPlus, 2, 1);  
  
        scene = new Scene(grid, 200, 100);  
        show();  
    }  
  
    public void show() {  
        stage.setTitle("Beispiel");  
        stage.setScene(scene);  
        stage.show();  
    }  
}
```





Properties: Beispiel

```
public class ViewModel {  
    public ViewModel(View view, Raum raum) {  
        view.tempLabel.setText(  
            String.valueOf(raum.getTemp()));  
        // Label erhält initiale Temperatur  
        // Listener an Temp-Property registrieren  
        raum.tempProperty().addListener(  
            new ChangeListener<Object>() {  
                @Override  
                public void changed(ObservableValue<?>  
                    wert, Object alt, Object neu) {  
                    // Label reagiert auf Temp.änderung  
                    view.tempLabel.setText(  
                        String.valueOf(raum.getTemp()));  
                }  
            });  
    }  
}
```



```
// Event-Handler an +-Button registrieren  
view.buttonPlus.setOnAction(new  
    EventHandler<ActionEvent>() {  
        @Override  
        public void handle(ActionEvent event) {  
            raum.setTemp(raum.getTemp() + 1);  
        }  
    });  
  
// Event-Handler an - -Button registrieren  
view.buttonMinus.setOnAction(new  
    EventHandler<ActionEvent>() {  
        @Override  
        public void handle(ActionEvent event) {  
            raum.setTemp(raum.getTemp() - 1);  
        }  
    });  
}
```





Properties: Beispiel

```
public class RaumtempApp extends Application {  
    @Override  
    public void start(Stage primaryStage) {  
        Raum wohnzimmer = new Raum();  
        wohnzimmer.setTemp(20.0);  
        View view = new View(primaryStage);  
        new ViewModel(view, wohnzimmer);  
    }  
  
    public static void main(String[] args) {  
        launch(args);  
    }  
}
```





Bindungen

- JavaFX-Properties können an andere JavaFX-Properties gebunden werden:
„Property Binding“
 - ermöglicht Synchronisation von Werten ohne Listener
 - z.B. soll Property a stets den Wert von Property b (i.a. desselben Typs) haben
 - **Unidirektionales** Binding: Änderung von a hat Auswirkung auf b, aber nicht umgekehrt
 - `b.bind(a)`
 - **Bidirektionales** Binding: Änderung von a hat Auswirkung auf b und umgekehrt
 - `b.bindBidirectional(a)`
- Neben der direkten Bindung von Properties kann auch die `Bindings`-Klasse und deren statische Methoden genutzt werden



Bindungen: Beispiel

- Anwendung mit Labels, Textfelder und Comboboxen
- Bindungen:
 - Wenn Netto-Textfeld (nettoInput) leer, dann sollen Brutto-Label (bruttoLabel) und Brutto-Textfeld (bruttoOutput) deaktiviert sein
 - Das Brutto-Textfeld enthält den Bruttowert (Nettowert + Steuer aus der Combobox cbUmsatzsteuer)

Brutto-Rechner 1

Netto: 40.0

Steuer: 19.0

Brutto: 47.6

Brutto-Rechner 1

Netto:

Steuer: 19.0

Brutto: 0.0



Bindungen: Beispiel



```
public class BruttoRechnerViewModel {

    public BruttoRechnerViewModel(BruttoRechnerView view) {
        view.bruttoLabel.disableProperty().bind(view.nettoInput.textProperty().isEmpty());
        view.bruttoOutput.disableProperty().bind(view.nettoInput.textProperty().isEmpty());

        StringConverter<Double> converter = new StringConverter<Double>() {
            @Override
            public Double fromString(String arg0) {
                Double d = 0d;
                try { d = Double.valueOf(arg0); } catch (Exception e) { }
                return d;
            }
            @Override
            public String toString(Double arg0) {
                if (arg0 == null) { return ""; }
                return arg0.toString();
            }
        };

        TextFormatter<Double> formatter = new TextFormatter<Double>(converter);
        view.nettoInput.setTextFormatter(formatter);

        DoubleProperty nettoprop = new SimpleDoubleProperty();
        nettoprop.bind(formatter.valueProperty());
        DoubleProperty taxprop = new SimpleDoubleProperty();
        taxprop.bind(view.cbUmsatzsteuer.valueProperty());

        NumberBinding brutto = Bindings.add(nettoprop, Bindings.multiply(nettoprop, taxprop).divide(100));
        view.bruttoOutput.textProperty().bind(brutto.asString());
    }
}
```



Programmieren

10. Graphische Benutzeroberflächen

10.6 FXML



FXML: Einführung

- FXML: **XML-basierte Markup-Sprache** z.B. für JavaFX-Scene Graphen von GUIs
 - Anwendung insbesondere bei UIs mit großen, komplexen Scene Graphen, Formularen, Eingaben, komplexe Animationen, dynamische Layouts mittels Skriptsprachen
- Anwendung folgt dem MVC-Entwurfsmuster
 - Nutzeroberfläche bleibt getrennt von Model und Controller
- **View**: FXML-Datei, in der die GUI-Oberfläche (Scene Graph) beschrieben wird
 - Vollständiger Baum aller graphischen Objekte
 - Viele JavaFX-Klassen können als Elemente und Bean Properties als Attribute verwendet werden
 - Generierung mittels XML- oder Texteditor oder mit SceneBuilder (spezieller Editor)
- **Controller**: Java-Klasse, in der z.B. die Event-Handler implementiert sind und das gesamte Verhalten der Oberfläche implementiert wird.
 - Implementiert oft das Initializable-Interface mit der initialize- Methode
- **Modell**: Java-Klasse, die die Anwendung (Domänenobjekte) enthält.
 - Wird über den Controller mit der View verbunden
- Siehe z.B. https://docs.oracle.com/javafx/2/get_started/fxml_tutorial.htm,
https://docs.oracle.com/javafx/2/fxml_get_started/why_use_fxml.htm



FXML: Einführung

- Die GUI wird also in einer FXML-Datei beschrieben.
- Beispiel: Button
 - `<Button fx:id="button" onAction="#manageButton" prefWidth="100.0" text="Klick!" GridPane.columnIndex="0" GridPane.rowIndex="0" />`
- In der XML-Datei gibt man im obersten Layout mit `fx:controller` an, wo die Controller-Klasse sich befindet und verknüpft die Event-Handler mit `#onAction` an den jeweiligen Elementen.
- Die Darstellung von GUI-Elementen kann direkt aus dem Controller heraus beeinflusst werden, wenn man eine `fx:id` in der FXML-Datei definiert, mit der die Elemente im Controller angesprochen werden können.



FXML: Anwendung

- Die Beschreibung der GUI ist ausgelagert in die Datei BruttoRechner.fxml
- Parent ist Oberklasse aller Knoten, die Kinder haben können (typischerweise Layouts)
- Es können auch die CSS-Styles ausgelagert werden.

```
public class BruttoRechner extends Application {  
  
    @Override  
    public void start(Stage stage) throws Exception {  
        Parent root =  
            FXMLLoader.load(  
                getClass().getResource("BruttoRechner.fxml"));  
        Scene scene = new Scene(root);  
        scene.getStylesheets().add(  
            getClass().getResource("BruttoRechner.css")  
                .toExternalForm());  
  
        stage.setScene(scene);  
        stage.setTitle("Brutto-Rechner");  
        stage.show();  
    }  
  
    public static void main(String[] args) {  
        launch(args);  
    }  
}
```



FXML: Anwendung

- Beispiel einer FXML-Datei
- Angabe des Controllers als vollqualifizierter Name (`fx:controller`)
- `fx:id` wichtig, damit GUI-Elemente im Quellcode angesprochen werden können

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.Button?>
<?import javafx.scene.control.ComboBox?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.layout.GridPane?>

<GridPane prefHeight="250.0" prefWidth="400.0" hgap="20.0" vgap="20.0"
  xmlns="http://javafx.com/javafx/8.0.171" xmlns:fx="http://javafx.com/fxml/1"
  fx:controller="bruttorechner2.BruttoRechnerController">
  <children>
    <Label text="Netto" GridPane.columnIndex="0" GridPane.rowIndex="0"/>
    <TextField fx:id="tfNetto" GridPane.columnIndex="1" GridPane.rowIndex="0" />
    <Label text="Steuer" GridPane.columnIndex="0" GridPane.rowIndex="1" />
    <ComboBox fx:id="cbUmsatzsteuer" GridPane.columnIndex="1" GridPane.rowIndex="1" />
    <Label fx:id="bruttoLabel" text="Brutto" GridPane.columnIndex="0" GridPane.rowIndex="2" />
    <TextField id="brutto" fx:id="tfBrutto" GridPane.columnIndex="1" GridPane.rowIndex="2" />
  </children>
</GridPane>
```



FXML: Anwendung

- Der Controller stellt die Anwendungslogik und Ereignisbehandlung bereit
- Das Tag @FXML sorgt für passende Initialisierung der Elemente durch das Framework
- Über `initialize` kann die GUI vorbereitet und es können zum Beispiel Properties gebunden werden
- Zusätzlich kann die Datenhaltung in separaten Modell-Klassen erfolgen (vollständiges MVC Muster)

```
public class BruttoRechnerController 
    implements Initializable {

    @FXML
    private Label bruttoLabel;

    @FXML
    private TextField tfNetto;

    @FXML
    private ComboBox<Double> cbUmsatzsteuer;

    @FXML
    private TextField tfBrutto;

    private Double[] items = { 19d, 16d, 7d };

    @Override
    public void initialize(
        URL url, ResourceBundle bundle) { ...
}
```